Technische Universität Berlin

DAI-Labor
Fakultät IV für Elektrotechnik und Informatik
Ernst-Reuter-Platz 7
10587 Berlin

Bachelor Thesis

Action-Label-Free World-Model Planning: Extending DINO-WM with Inverse Dynamics

Julian Quast

Matriculation Number: 450764

15.02.2026

Supervised by
Dr.-Ing. Stefan Fricke

Second Reviewer
Prof. Dr. Stefan Tai

Eidesstattliche Erklärung

Hiermit versichere ich, dass ich die vorliegende Arbeit selbstständig und ohne Hilfe Dritter angefertigt habe und dabei ausschließlich die angegebenen Quellen und Hilfsmittel verwendet habe. Alle Stellen, die wörtlich oder sinngemäß aus diesen Quellen entnommen sind, habe ich als solche kenntlich gemacht.

Sofern generative KI-Werkzeuge eingesetzt wurden, habe ich Produktnamen, Hersteller, die jeweils verwendeten Softwareversionen sowie die konkreten Einsatzzwecke (z. B. sprachliche Überprüfung und Verbesserung der Texte, Unterstützung bei der systematischen Recherche) transparent benannt. Die Auswahl, Übernahme und sämtliche Ergebnisse des von mir verwendeten KI-generierten Outputs verantworte ich vollumfänglich selbst.

Zur grammatikalischen Überprüfung, stilistischen Überarbeitung sowie zur Unterstützung bei der systematischen Recherche habe ich das generative KI-Werkzeug ChatGPT (OpenAI, Version 5.2) verwendet. Zur gemeinsamen Entwicklung und Implementierung habe ich ChatGPT (OpenAI, Version 5.2, Codex) verwendet. Die Verantwortung für die korrekte Anwendung dieser Werkzeuge sowie für die finale Fassung sämtlicher Inhalte liegt ausschließlich bei mir.

Die Satzung zur Sicherung guter wissenschaftlicher Praxis an der Technischen Universität Berlin vom 15. Februar 2023 habe ich zur Kenntnis genommen. Weiterhin erkläre ich, dass ich diese Arbeit in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegt habe.

Berlin, 15.02.2026

.....

(Unterschrift)

Abstract

World models enable agents to predict the consequences of their actions and plan in imagined futures. Recent work combines large-scale self-supervised visual encoders with action-conditioned predictors, demonstrating strong generalization and even zero-shot planning. However, most approaches still rely on labeled action data, which is scarce and embodiment-specific, limiting scalability and transfer. Although latent action models aim to learn control representations directly from observations, they often depend on pixel-level reconstruction and use reactive policies instead of test-time optimization. In this work, we study whether model-based planning performance remains comparable when action labels are replaced by discrete latent actions. We present a world model trained from offline image-only trajectories that infers discrete latent actions via inverse dynamics in DINOv2 embedding space and jointly learns a forward dynamics predictor. Goal conditioned planning is performed through trajectory optimization over the learned latent action space using discrete CEM. On PushT, we show that a continuous latent-action formulation achieves low latent prediction error but collapses during planning (SR 0.02). Introducing vector quantization prevents shortcut learning and enables non-trivial planning (SR 0.19), yet remains below the action-supervised baseline (SR 0.52). These results indicate that low prediction error alone does not guarantee controllable dynamics, and that action-label-free planning requires a latent-action bottleneck together with improved alignment between learned action codes, long-horizon rollouts, and downstream decoding.

Zusammenfassung

World Models ermöglichen es *agents*, die Konsequenzen ihrer *actions* vorherzusagen und dadurch in ihren Imaginationen zu planen. Im Rahmen einschlägiger Arbeiten werden große *self-supervised encoder* mit leistungsfähigen aktionskonditionierten Dynamikvorhersagemodellen kombiniert. Diese Modelle sind umgebungs- und taskunspezifisch und zeigen Zero-Shot-Planning-Fähigkeiten. Ein Großteil der verwendeten Ansätze basiert auf gelabelten Datensätzen. Das hat zur Folge, dass solche Modelle nicht auf ungelabelten Videodaten trainiert werden können und robotikspezifischen Aktionsrepräsentationen nutzen. Dadurch werden Skalierbarkeit und Transfermöglichkeiten begrenzt. Modelle, in denen Aktionsrepräsentationen direkt aus Videodaten erlernt werden, adressieren benanntes Problem. Diese basieren jedoch häufig auf generativen Ansätzen und erlernen reaktive *Policies*, anstatt das gelernte *World Model* für Test-Time-Optimierung zu nutzen. In der vorliegenden Arbeit wird untersucht, ob mit der Nutzung von selbst erlernten Aktionsrepräsentationen gleichwertige Planungsleistungen erzielt werden können. Es wird ein *World Model* vorgestellt, welches ausschließlich auf Videodaten trainiert wird und latente *actions* mittels eines *inverse dynamics model*, gemeinsam mit dem Dynamikvorhersagemodell im DINOv2 Repräsentationsraum erlernt. Es wird eine diskrete Variante der Cross-Entropy-Method (CEM) für zielbasiertes Planen herangezogen. Auf dem PushT-Benchmark zeigt die untersuchte Variante ohne Vektorquantisierung niedrige Vorhersagefehler aber nur eine Erfolgsrate (SR) von 2 %. Die Integration von Vektorquantisierung verhindert effektiv *shortcut learning*, erzielt eine Erfolgsrate von 19%, bleibt jedoch unter der Baseline (SR 0.52). Die Ergebnisse deuten darauf hin, dass geringe Vorhersagefehler nicht zwangsläufig kontrollierbare Dynamiken implizieren. Erfolgreiches Planen mit latenten *actions* erfordert eine sorgfältig abgestimmte Informationsbeschränkung

sowie Konsistenz zwischen latenten *actions*, längerer Rollouts und dem Dekodieren zurück zu den *ground-truth action labels*.

Contents

List of Figures vii

List of Tables viii

1 Introduction 1

2 Background 2

2.1 Problem Setting . . . 2
2.2 World Models . . . 3
2.3 Self-supervised Pretrained Visual Representations . . . 4
2.4 Joint-Embedding Predictive Architectures . . . 5
2.5 Latent Action Models . . . 5
2.6 Vector Quantization Bottlenecks . . . 7
2.7 Planning in Discrete Latent Action Space . . . 8

3 Related Work 9

3.1 Policy Learning with Model-based RL . . . 10
3.2 Latent Action Policy Learning . . . 11
3.3 Planning with Model-based RL . . . 12
3.4 Planning in Latent Action Space . . . 13
3.5 Joint-Embedding Predictive World Models . . . 13
3.6 Thesis Positioning . . . 14

4 DINO World Model with Inverse Dynamics 15

4.1 Overview . . . 15
4.2 Encoder . . . 16
4.3 Inverse Dynamics Model . . . 17
4.4 Forward Dynamics Model . . . 18
4.5 Training Procedure . . . 19
4.6 Visual Planning . . . 21
4.7 Latent Action Decoder . . . 22

5 Experimental Setup 22

5.1 Overview . . . 22
5.2 Diagnostics of Failure Modes . . . 23
5.3 Environment and Dataset . . . 24
5.4 Implementation . . . 26
5.5 Evaluation Protocol . . . 34
5.6 Experiments . . . 34

6 Results 35

6.1 Prediction Quality . . . 36
6.2 Action Decoding Quality . . . 36

6.3 Planning Performance 37
6.4 Latent Action Diagnostics 38

7 Discussion **38**
7.1 Baseline as a reference 39
7.2 Low prediction error via a shortcut does not yield controllable dynamics 39
7.3 Vector quantization prevents leakage but weakens long-horizon fidelity 40
7.4 Why planning remains below the baseline even when latent actions are stabilized 41
7.5 Discrete latent actions behave like policy inputs, not standalone action labels . . 41
7.6 Limitations 41
7.7 Future Work 42

8 Conclusion **44**

A Appendix **49**
A.1 Implementation and Integration Effort 49

List of Figures

1 Overview of the proposed architecture. A frozen encoder maps observations to latent representations. The inverse dynamics model infers a latent action, which is subsequently quantized by a vector-quantization bottleneck. The forward dynamics model then predicts the next latent representation. The model is trained using a prediction loss for the dynamics model, while the inverse dynamics model is optimized using the same prediction loss augmented with an additional VQ commitment loss. Example input images from (Zhou et al., 2024) while the figure is inspired by (Assran et al., 2025). 15

2 Overview of the visual planning procedure. A frozen visual encoder maps observations to latent representations. Candidate sequences of discrete latent actions are optimized by rolling them out in the learned forward dynamics model and selecting sequences whose predicted terminal latent states best match the goal representation. Example input images and inspiration from (Zhou et al., 2024). 21

3 Example observation from a training trajectory illustrating the PushT environment. Sampled fron the PushT noise dataset (Zhou et al., 2024). 25

4 Active model components and configuration parameters for the Latent-Action VQ variant during training. Rounded boxes on the left denote configuration flags. Rectangular boxes represent model components. Hatched elements indicate frozen components. Violet highlights denote active components and flags, while light grey indicates disabled or unused components and flags. 30

5 Active model components and configuration parameters for the Latent-Action VQ variant during planning. Rounded boxes on the left denote configuration flags. Rectangular boxes represent model components. Hatched elements indicate frozen components. Violet highlights denote active components and flags, while light grey indicates disabled or unused components and flags. 30

6 Active model components and configuration parameters for the latent-action variant during training.Rounded boxes on the left denote configuration flags. Rectangular boxes represent model components. Hatched elements indicate frozen components. Violet highlights denote active components and flags, while light grey indicates disabled or unused components and flags. 31

7 Active model components and configuration parameters for the latent-action variant during planning. Rounded boxes on the left denote configuration flags. Rectangular boxes represent model components. Hatched elements indicate frozen components. Violet highlights denote active components and flags, while light grey indicates disabled or unused components and flags. 31

8 Active model components and configuration parameters for the baseline during training. Rounded boxes on the left denote configuration flags. Rectangular boxes represent model components. Hatched elements indicate frozen components. Violet highlights denote active components and flags, while light grey indicates disabled or unused components and flags. 32

9 Active model components and configuration parameters for the Baseline during planning. Rounded boxes on the left denote configuration flags. Rectangular boxes represent model components. Hatched elements indicate frozen components. Violet highlights denote active components and flags, while light grey indicates disabled or unused components and flags. 33

List of Tables

1 Summary of related work. Columns indicate whether a method performs inference-time planning, uses inferred latent actions instead of ground truth labels, learns forward dynamics in latent space, and relies on a pretrained encoder. 10
2 Vision Transformer predictor architecture. 26
3 Training and model configuration parameters used in all experiments. 27
4 Overview of model variants and their active components. IDM denotes the inverse dynamics model. Used action types: [1] embedded ground truth action, [2] continuous latent action, [3] quantized action vector from the codebook. 28
5 Variant specific training parameters of the Latent Action VQ Model 28
6 Variant specific planning parameter of the Latent Action VQ Model 29
7 Inverse Dynamics Model Implementation Details 29
8 Variant specific training parameters of the Latent Action Model. 30
9 Variant specific planning parameters of the Latent Action Model. 31
10 Baseline configuration parameters (training-time). 32
11 Baseline planning configuration parameters (planning-time). 32
12 Fixed planning and evaluation configuration used in all planning experiments. . . 33
13 Training and evaluation configuration for the Action Decoder. 34
14 Prediction Quality on the Validation Set. Next-Step MSE evaluates single-step latent prediction accuracy. Rollout MSE measures open-loop latent prediction error over 600 seeded random-length rollouts. Values are reported as mean ± standard deviation. We highlight the best results. 36
15 Action decoding performance (RMSE) of the LA variant on the validation split for different decoder input configurations and training set sizes. Lower is better. A mean-action baseline achieves an RMSE of 1.44. 36
16 Action decoding performance (RMSE) of the LA-VQ varaint on the validation set for different decoder input configurations and training set sizes. Lower is better. A mean-action baseline achieves an RMSE of 1.44. 37
17 Planning performance of all variants for the last iteration, reported as the mean over 100 different evaluation tasks drawn from the validation split. SR denotes the fraction of successful trajectories. StateDist is the mean ℓ_2 distance between the final and goal states in ground-truth state space. PixL2 is the mean ℓ_2 distance between final and goal observations in pixel space. 37
18 Latent action diagnostics of the final models on the validation set. **z-swap** reports the fraction of swaps where the resampled latent action code effectively selects another sample's next state, i.e., $f(z_t^{(i)}, u_t^{(j)})$ is closer to $z_{t+1}^{(j)}$ than to $z_{t+1}^{(i)}$. $\Delta_{\textbf{shuf}}$ reports the factor by which the prediction error increases when actions are shuffled across samples. PPL_{norm} is the normalized codebook perplexity, while **Dead rate** is the fraction of codebook entries that are never selected. 38

1 Introduction

World models play an increasingly important role in modern embodied AI (Ha et al., 2018; Hafner et al., 2023; Zhou et al., 2024; Bruce et al., 2025; Assran et al., 2025; Pu et al., 2025). They enable agents to internally predict the consequences of their actions, allowing them to plan and learn policies based on imagined future trajectories. In parallel, driven by recent advances in large-scale self-supervised representation learning—grounded in transformer architectures (Vaswani et al., 2017) and foundation-model pretraining—embodied AI is increasingly adopting general-purpose visual encoders such as DINOv2 (Oquab et al., 2023). These models show promising results on several downstream control tasks and in addressing generalization challenges (Zhou et al., 2024; Assran et al., 2025; Bu et al., 2025). With DINO-WM, Zhou et al., 2024 introduced a world model that combines a pretrained DINOv2 encoder with an action-conditioned predictor. This architecture performs planning directly in the embedding space and demonstrates zero-shot planning capabilities on multiple benchmarks.

Despite these advances, most state-of-the-art models still require labeled interaction data to learn action-conditioned dynamics and control policies (Hafner et al., 2023; Hansen et al., 2023; Zhou et al., 2024; Assran et al., 2025). Since internet-scale action-labeled datasets are scarce and costly to acquire, while raw video is abundant, this reliance on labeled actions limits the scalability of world models compared to purely visual pretraining. Moreover, a truly foundational world model should operate across diverse environments and embodiments. This would require a unified action representation abstracting over embodiment-specific control interfaces, which is often impractical or ill-posed, as action semantics are tightly coupled to physical structure and actuation. As a result, reliance on explicit action labels further limits generalization and transfer (Bu et al., 2025). In response to this limitation, current work on latent action models learns implicit action representations from observations alone, typically via inverse dynamics. These models encode the changes between consecutive observations into a compact latent action, most often using a VAE- or VQ-VAE–based approach (Kingma et al., 2013; Oord et al., 2017; Bruce et al., 2025; Gao et al., 2025). Such latent actions can then serve as a control interface for downstream tasks, and several works have demonstrated successful planning and policy learning using these representations (Schmidt et al., 2023; Bu et al., 2025; Ye et al., 2024a; Zhang et al., 2024; Gao et al., 2025; Bruce et al., 2025). However, these methods often rely on pixel-space reconstruction losses rather than learning directly in embedding space, and they usually train the latent action model separately from the world model or policy. This separation also means that decoders are discarded during downstream use, disconnecting learned latent actions from the planning and control components (Gao et al., 2025; Bruce et al., 2025). Furthermore, many latent action approaches focus on training policies rather than enabling explicit model-based planning. Once the policy is trained, incorporating new task information usually requires retraining the entire system, whereas learning a dynamics model would allow task optimization at inference time without additional training (Bu et al., 2025; Schmidt et al., 2023; Ye et al., 2024a).

In this thesis, we investigate whether the benefits of pretrained visual embeddings and model-based planning can be combined with latent action learning. Concretely, we build on a publicly available, pretrained visual encoder and propose a variant of DINO-WM that jointly learns a

forward dynamics model and an inverse dynamics model directly in the learned embedding space. This design enables pretraining the model without ground-truth action labels while reducing the number of learned components. The forward model predicts future DINO embeddings conditioned on latent actions, while the inverse dynamics model maps pairs of consecutive embeddings to latent actions. Training couples these components so that inferred latent actions are directly optimized for forward prediction. Planning is performed in the latent action space using a CEM optimizer (Rubinstein, 1999). This motivates the central research question of this thesis: Can our proposed latent action version of DINO-WM, trained with joint forward and inverse dynamics, match the planning performance of the original model on the PushT benchmark?

To answer this question, we review inverse dynamics architectures and use the resulting insights to propose our variant of DINO-WM. We then empirically analyze how the introduction of latent actions into the pipeline affects its behavior. In particular, we analyze and compare prediction quality, planning performance, and the effect of actions and latent actions on prediction for our variants relative to the baseline. We additionally evaluate the decodability of the learned latent actions into ground-truth action labels. This thesis is structured as follows. We first provide the scientific background for our approach and review related work and alternative methods. We then introduce our proposed architecture in detail. Next, we describe our experimental setup, covering dataset curation, the chosen benchmark, training and implementation details, and the conducted experiments. We then present and discuss our empirical results. Finally, we summarize the contributions and limitations of our approach and outline promising directions for future work.

2 Background

2.1 Problem Setting

We adopt the standard formulation of partially observable Markov decision processes (POMDPs) as introduced by Kaelbling et al. (1998). Under this formulation, the environment in which an agent acts is modeled as a POMDP, defined by the tuple $(\mathcal{S}, \mathcal{A}, \Omega, T, \mathcal{O}, R)$, where $\mathcal{S}$ denotes the true underlying state space of the environment, which is generally unobserved and unknown to the agent. $\mathcal{A}$ denotes the action space and Ω denotes the observation space, with $o_t \in \Omega$ denoting an observation available to the agent. At each discrete time step t, the environment is in state $s_t \in \mathcal{S}$, and the agent selects an action $a_t \in \mathcal{A}$. The environment dynamics are governed by the stochastic transition function $T(s_{t+1} \mid s_t, a_t)$, which specifies the conditional probability of transitioning to the next state s_{t+1} given the current state s_t and action a_t. The agent receives an observation $o_{t+1} \sim O(\cdot \mid s_{t+1})$, where $\mathcal{O}(\cdot \mid \cdot)$ defines the distribution over observations conditioned on the current state.

In this thesis, we focus on visual world modeling and therefore restrict observations to image streams $o_t = x_t$. While the POMDP formulation typically includes a reward function R, and in some definitions also a discount factor γ and an initial belief b_0, our method does not model rewards, optimize a return, or rely on an initial belief. These components are introduced solely to align notation with prior model-based reinforcement learning literature.

In practice a learned model of the environment is conditioned on the interaction history h_t, where h_t is typically aggregated into a latent belief state using a recurrent state estimator (Hafner et al., 2019). More recent transformer-based architectures instead maintain a finite history of latent representations and actions as the model's internal state for predicting future latents (Zhou et al., 2024; Chen et al., 2022).

2.2 World Models

A world model is a learned system that predicts how the environment's observable dynamics evolve over time, conditioned on the agent's actions (Ha et al., 2018; Hafner et al., 2022). In this thesis, we focus on visual, latent-space world models, i.e., models learned from image observations $o_t = x_t$ that operate in a learned latent space rather than directly in observation space. While alternative formulations exist, the components introduced here are chosen to reflect the class of world models most relevant to this thesis. Such models can be decomposed into an encoder, a forward dynamics model, often referred to as a predictor, and optionally a decoder (Zhou et al., 2024).

The encoder is defined as $\text{enc}_\psi : \Omega \to \mathcal{Z}$, mapping raw observations to a latent representation as $z_t = \text{enc}_\psi(o_t)$. In many recent works, the encoder is typically frozen after self-supervised pretraining, and only the dynamics components are trained for the downstream domain(Zhou et al., 2024; Assran et al., 2025; Oquab et al., 2023). Actions may likewise be embedded before being consumed by the dynamics model. The forward dynamics model is represented as $\hat{z}_{t+1} = f_\theta(z_{t-L:t}, a_{t-L:t})$, where L is the context length, and learns to predict future latent representations from past latent representations and actions, thereby approximating environment dynamics in latent space. By conditioning predictions on actions, world models aim to capture how agent interventions influence future observations, rather than merely modeling passive temporal correlations. This capability is required for policy learning and planning, and it reduces ambiguity over possible future outcomes. A decoder maps latent states back to observation space, $g_\phi : \mathcal{Z} \to \Omega$, and defines $\hat{o_{t+1}} = g_\phi(z_{t+1})$. While useful for visualization, such decoders are optional in latent-space world models (Zhou et al., 2024).

Each component of the model can be trained using different learning objectives. A common approach for training the encoder is reconstruction-based learning, where the encoder is optimized to produce latent codes that preserve sufficient information to reconstruct the input observation (Oquab et al., 2023). Alternatively, in predictive representation learning, the encoder is trained to produce representations that are informative for predicting missing or future observations, for example via masked prediction or next-step prediction objectives (Assran et al., 2023). In this setting, the latent codes are optimized to support accurate forecasts by a predictor model rather than faithful reconstruction of the input. The forward dynamics model is trained to minimize a discrepancy between predicted and target future latent representations using latent-space losses. Decoders, when present, are typically trained using reconstruction losses defined with respect to the target image.

World models can support control in two principal ways. First, in planning, the world model is used online at inference time to optimize action sequences under a reward or goal objective

(Zhou et al., 2024). Second, in policy learning, a world model is used as a simulator to train a reactive policy , either entirely inside the model or by alternating between model learning and data collection (Hafner et al., 2019; Kaiser et al., 2019).

A key limitation of world models is compounding prediction error, which can degrade both policy learning and planning when learned dynamics deviate from the true environment (Talvitie, 2016). This issue is especially critical for long-horizon planning, motivating careful choices of representation, horizon length, and optimization strategy.

2.3 Self-supervised Pretrained Visual Representations

Visual representations are learned feature encodings that map high-dimensional pixel observations, such as images or video frames, into compact latent vectors. The goal is to preserve task-relevant structure—such as object identity, spatial layout, and semantics—while discarding nuisance variation, including sensor noise, minor illumination changes, or small viewpoint changes. These latent vectors should be general-purpose features that perform well across domains and downstream tasks, including classification, semantic segmentation, and depth estimation (Oquab et al., 2023; Assran et al., 2023; Bengio et al., 2013a).

While convolutional neural networks (CNNs) historically played a central role due to inductive biases such as locality and translation invariance (He et al., 2015), Vision Transformers (ViTs) have emerged as a popular alternative for large-scale pretraining, as they scale effectively with data and model long-range dependencies via self-attention (Dosovitskiy et al., 2020). ViTs represent an image by splitting it into fixed-size patches, flattening each patch, and projecting it into an embedding vector, thereby forming a sequence of learned tokens $\{t_1, \ldots, t_N\}$ augmented with positional embeddings. These patch-token outputs provide dense, spatially localized features. To obtain a compact global image representation, many architectures prepend a learned special token [CLS], whose final hidden state serves as a summary of the patch embeddings.

Self-supervised learning (SSL) trains such encoders without manual human labeling by deriving supervision from the data itself, encompassing automated curation pipelines, contrastive methods, masked image modeling, teacher–student distillation approaches, and predictive representation learning methods (Chen et al., 2020a; Caron et al., 2021; He et al., 2021; Assran et al., 2023).

DINOv2 is a prominent discriminative self-supervised learning method that learns visual representations from large-scale curated image data using a teacher–student distillation framework. It builds on earlier approaches such as DINO (Caron et al., 2021) and iBOT (Zhou et al., 2021) and incorporates centering mechanisms inspired by SwAV (Caron et al., 2020). During training, a student network is encouraged to match the representations produced by a slowly evolving teacher network, updated as an exponential moving average (EMA) of the student, across multiple strongly augmented views of the same image. This view-consistency objective induces invariances that support the emergence of semantically structured representations.

Empirical evaluations demonstrate that the resulting patch token embeddings achieve competitive performance across many downstream tasks, often rivaling or surpassing weakly supervised approaches. These results highlight that self-supervised pretraining alone can produce strong

and transferable visual representations that can be used out of the box as frozen encoders across domains (Oquab et al., 2023).

While such representations provide a strong foundation, task-specific fine-tuning can yield small additional performance gains. At the same time, these representations are primarily optimized for semantic visual understanding rather than for explicitly encoding 3D geometry or physical dynamics. The invariances induced during self-supervised training, while beneficial for robustness and transferability, may also attenuate fine-grained visual details. Finally, the training of large Vision Transformer models typically entails substantial computational cost, motivating their reuse as pretrained backbones (Oquab et al., 2023).

In world models, pretrained encoders naturally serve as perception front-ends that map observations o_t to latent states on which dynamics predictors, sequence models, and planners operate, thereby improving sample efficiency by shifting learning from pixels to latent representations. Furthermore, patch-token features can support attention-based, object- or region-level dynamics modeling, which can benefit fine-grained control in embodied settings (Zhou et al., 2024).

2.4 Joint-Embedding Predictive Architectures

Joint-Embedding Predictive Architectures (JEPA) learn by predicting in representation space rather than pixel space. Given two consecutive video frames x_t and x_{t+1}, two encoders map them to latent representations $z_t = \text{enc}_\psi(x_t)$ and $z_{t+1} = \text{enc}_{\psi'}(x_{t+1})$, which do not need to share parameters and may even process different modalities. A predictor f_θ is then trained to produce a representation-level prediction $\hat{z}_{t+1} = f_\theta(z_t, c)$, optionally conditioned on an additional latent variable c. Learning minimizes a distance $d(z_{t+1}, \hat{z}_{t+1})$, so the model captures dependencies between x_t and x_{t+1} without needing to generate every detail of x_{t+1}. This abstraction is particularly useful in settings such as video, where many details are inherently hard to predict and can be intentionally abstracted away by the encoder. Multi-modality can be represented either through invariances of the encoder, where multiple x_{t+1} map to similar z_{t+1}, or through the conditioning variable c, which can index multiple plausible predictions. In practice, JEPA-style training is frequently paired with non-contrastive objectives such as VICReg (Bardes et al., 2021), Barlow Twins (Zbontar et al., 2021), or SimSiam (Chen et al., 2020b), which prevent representation collapse by encouraging informative encodings of x_t and x_{t+1}. In action-conditioned settings, the action can be incorporated into the conditioning information as an instance of c, for example $\hat{z}_{t+1} = f_\theta(z_t, a_t)$, so that different actions yield different predicted future latent states (LeCun, 2022; Assran et al., 2023).

DINO-WM (Zhou et al., 2024) can be interpreted as a JEPA-style world model in the sense that it predicts future latent embeddings in representation space conditioned on actions, optimizing an embedding-space prediction objective rather than pixel-level next-frame reconstruction.

2.5 Latent Action Models

Latent Action Models (LAMs) aim to represent control not by an explicit, predefined action from the environment's native action space, but by a learned latent variable that explains how

observations change over time. Conceptually, latent actions are treated as unobserved causes of state transitions. Instead of assuming that an action label is available for each transition, the model infers a compact action representation directly from consecutive observations (Edwards et al., 2018; Pathak et al., 2017; Gao et al., 2025). This is practically motivated by the fact that large-scale video demonstrations are abundant, whereas action-labeled interaction datasets are scarce, expensive, or infeasible to collect at comparable scale. Furthermore, when designing multi-environment or embodied models, it is often impractical—and in some cases arguably ill-posed—to merge the action spaces of multiple environments into a single unified representation (Bu et al., 2025). LAMs therefore enable training on passive observation sequences and abstract action spaces, while still learning transition-relevant structure that can support downstream control.

A common formulation couples an inverse dynamics model (IDM) with a forward dynamics model (FDM), operating either directly in observation space or in a learned latent embedding space (Schmidt et al., 2023; Bu et al., 2025). Let z_t denote a latent representation of the observation at time t. The inverse model infers a latent action from a transition,

$$\tilde{u}_t = q_\omega(z_t, z_{t+1})$$

and the forward model predicts the next latent representation conditioned on this inferred action,

$$\hat{z}_{t+1} = f_\theta(z_t, \tilde{u_t}).$$

Training typically minimizes a prediction loss in latent space, encouraging the inferred action to encode exactly the information required to predict the transition beyond what is already present in the current latent state z_t.

A central technical challenge in this setup is avoiding an information bypass (Gao et al., 2025; Schmidt et al., 2023). Without a suitable information bottleneck, the system can minimize the loss by allowing $\tilde{u}_t$ to carry all information about z_{t+1}, such that f_θ learns a trivial decoder,

$$f_\theta(z_t, \tilde{u_t}) \approx \tilde{u_t}.$$

Common choices include strict dimensionality constraints, stochastic latents with KL regularization as in variational autoencoders (Kingma et al., 2013), and discrete latents via vector quantization (Oord et al., 2017), where u_t is restricted to a finite codebook. The bottleneck encourages latent actions to capture compact, reusable, action-like abstractions rather than transition-specific encodings. For downstream control, latent actions must ultimately interface with decision making and, in many cases, be related back to the environment's native action space. In policy-learning settings, a policy can be trained directly to output latent actions, optionally followed by a learned decoder or policy head that maps them to executable environment actions. In planning-centric settings, the learned dynamics model can be rolled out under candidate latent action sequences and optimized at decision time, enabling task adaptation without retraining when goals change. When interaction with the environment is required, the mapping from latent

to native actions is commonly learned via fine-tuning with a comparatively small amount of interaction or action-labeled data (Gao et al., 2025; Nikulin et al., 2025; Bu et al., 2025).

2.6 Vector Quantization Bottlenecks

Vector quantization (VQ) introduces an explicit information bottleneck by strictly limiting the representational capacity of latent variables, thereby preventing the model from encoding arbitrary continuous detail. This capacity restriction is induced by mapping a continuous latent space onto a finite set of discrete representations (Oord et al., 2017). As a result, similar latent embeddings are collapsed onto shared prototypes, which promotes abstraction and encourages the reuse of latent representations across different contexts (Razavi et al., 2019). Since the quantized latent can take on only K distinct values, the information capacity of the bottleneck is explicitly bounded by $\log K$ (in bits or nats, depending on the logarithm). These properties are particularly desirable in architectures whose goal is to learn compact representations that capture higher-level structure rather than low-level variation. Accordingly, several prior works have demonstrated that VQ bottlenecks can be successfully integrated into a wide range of such model architectures (Oord et al., 2017; Bu et al., 2025).

While vector quantization can be applied to a variety of latent variables, we focus on its use for latent actions, as this is most relevant to the objectives of this thesis. In the context of latent action modeling, the bottleneck is crucial because it prevents the latent action u from representing static visual or state information and instead forces it to encode abstract, transition-relevant structure (Schmidt et al., 2023).

Given a continuous latent action embedding $\tilde{u} \in \mathbb{R}^D$ and a codebook $E = \{u_1, \ldots, u_K\}$ with $u_k \in \mathbb{R}^D$, the quantization operation selects the nearest codebook entry according to the Euclidean distance:

$$k = \arg\min_j \|\tilde{u} - u_j\|_2^2,$$

$$u = u_k.$$

Downstream components then operate on the quantized codebook vectors u rather than directly on the discrete codebook indices. Therefore, the continuous embedding $\tilde{u}$ is replaced by u.

Since the nearest-neighbor assignment is non-differentiable, gradients are propagated using a straight-through estimator, which treats the quantization operation as the identity function during backpropagation (Bengio et al., 2013b). Compared to continuous bottlenecks such as Gaussian latent variables, vector quantization enforces a fixed discrete vocabulary that provides hard capacity control, prevents continuous representational drift, promotes explicit reuse of identical latent symbols and stabilizes latent semantics across contexts (Oord et al., 2017).

A key challenge in vector quantization is maintaining a stable and well-utilized codebook. Without appropriate stabilization mechanisms, some codebook entries may become underutilized or inactive (dead codes), effectively reducing the capacity of the discrete latent space.

Standard gradient-based codebook updates learn the codebook via backpropagation and include a dedicated codebook loss that moves codebook vectors toward encoder outputs:

$$\mathcal{L}_{\text{codebook}} = \|\text{sg}(\tilde{u}) - u\|_2^2,$$

where $\text{sg}(\cdot)$ denotes the stop-gradient operator. In this variant, $\mathcal{L}_{\text{codebook}}$ provides gradients for the codebook vectors (Oord et al., 2017).

EMA-based codebook updates replace direct gradient-based optimization with exponential moving averages of encoder assignments, thereby decoupling codebook adaptation from backpropagated gradients and often improving training stability (Oord et al., 2017). Concretely, for each codebook entry, the model maintains EMA statistics of the number of assignments and the sum of assigned encoder outputs, and updates the codebook vectors as normalized averages. The EMA dynamics are controlled by a decay hyperparameter $\gamma \in [0, 1)$. Larger γ yields slower, smoother updates, while smaller γ adapts faster but can track encoder outputs.

To ensure alignment between inferred continuous latent actions and the discrete codebook, a commitment loss is introduced (Oord et al., 2017):

$$\mathcal{L}_{\text{commit}} = \beta \, \|\tilde{u} - \text{sg}(u)\|_2^2,$$

where β controls the strength of the constraint. The commitment loss discourages the encoder from producing latent action embeddings that deviate arbitrarily from the discrete action space, thereby ensuring consistent codebook usage and preserving the interpretability of latent actions.

Finally, the codebook size K serves as an explicit capacity-controlling hyperparameter, smaller values of K enforce stronger compression and coarser action abstractions, while larger values allow for more detailed but less reusable latent action representations, increasing the risk that non–action-relevant information is encoded in the latent action.

2.7 Planning in Discrete Latent Action Space

Planning in latent action space refers to optimizing sequences of latent actions $u_0, \ldots, u_H$ under a predictive dynamics model. A standard instantiation of this approach is model-predictive control (MPC), which repeatedly solves a finite-horizon optimization problem under the current state estimate, executes only the first action, and replans at the next time step, resulting in a receding-horizon control loop. This formulation is widely used in model-based control because it supports flexible goal conditioning and online adaptation, and repeated replanning can mitigate the effects of compounding prediction errors and external disturbances (Zhou et al., 2024; Hafner et al., 2018; Hansen et al., 2023).

Various optimization methods have been proposed in the literature to solve the trajectory optimization problem arising in MPC. These include gradient-based optimization through differentiable dynamics models, random-shooting approaches such as MPPI (Williams et al., 2015; Williams et al., 2016), and other sampling-based methods. A commonly used sampling-based optimization method is the Cross-Entropy Method (CEM) (Rubinstein, 1999; Kroese et al., 2016), which has been used for world-model-based latent action planning (Zhou et al., 2024). While

CEM is traditionally formulated for continuous action spaces, it can be naturally adapted to discrete latent action spaces by replacing the continuous sampling distribution with a categorical one.

In the discrete variant of CEM, a sampling distribution is maintained over sequences of discrete latent actions of a fixed planning horizon H. This distribution is typically modeled as a factorized categorical distribution, parameterized by per-time-step probability vectors $\{\pi_t \in \Delta^{K-1}\}_{t=1}^{H}$, where K denotes the size of the discrete action vocabulary. At each iteration, N candidate action sequences are sampled from this distribution and evaluated by rolling them out through the learned dynamics model starting from an initial latent state z_0 under a given objective function.

The top-k action sequences with the highest objective values are then selected as elite samples. The categorical distribution parameters are updated by matching π_t to the empirical action frequencies of the elites at each time step, yielding a maximum-likelihood update under the elite set. This process is repeated for several iterations until convergence, resulting in a discrete action sequence that approximately maximizes the planning objective.

While various objective functions are possible, goal-directed objectives (Zhou et al., 2024)—such as minimizing the distance between the final predicted latent state $\hat{z}_g$ and a target goal embedding z_g—are most relevant for this thesis . When planning in a learned latent space, such distance-based goal objectives implicitly assume that distances in the embedding geometry correspond to meaningful progress toward the task goal, which may not always hold. Despite its strong performance, MPC with CEM requires a large number of model rollouts per decision step, trading increased planning flexibility for higher computational demand.

3 Related Work

This chapter reviews prior work most relevant to learning world models for control under limited supervision, with emphasis on how learned models are used at decision time and what supervision is available during training. Along the control axis, we distinguish methods that primarily execute a reactive policy at test time from methods that explicitly plan or optimize action sequences using a learned dynamics model. Along the supervision axis, we contrast interactive settings—where trajectories are labeled with actions and often rewards—with offline, observation-centric settings. In the latter, learning is driven by pre-recorded trajectories, and ground-truth conditioning actions may be replaced by learned latent actions. Table 1 summarizes the reviewed approaches, highlighting the absence of methods that combine inference-time planning with latent action dynamics learned in pretrained latent representation spaces.

Paper	Planning	Latent Actions	Latent FDM	Pretrained Encoder
World Models	✗	✗	✓	✗
DreamerV3	✗	✗	✓	✗
TransDreamer	✗	✗	✓	✗
ILPO	✗	✓	✗	✗
LAPO	✗	✓	✗	✗
UniVLA	✗	✓	✓	✓
TD-MPC2	✓	✗	✓	✗
MuZero	✓	✗	✗	✗
DINO-WM	✓	✗	✓	✓
AdaWorld	✓	✓	✗	✗
V-JEPA 2-AC	✓	✗	✓	✓

Table 1: Summary of related work. Columns indicate whether a method performs inference-time planning, uses inferred latent actions instead of ground truth labels, learns forward dynamics in latent space, and relies on a pretrained encoder.

We include canonical methods to establish shared baselines alongside recent work that directly relates to the design choices of this thesis. For each approach, we summarize the setting and core idea, then highlight implications for latent action planning in learned representation spaces.

3.1 Policy Learning with Model-based RL

We begin with approaches in which action selection at decision time is performed by a learned policy, including both model-based reinforcement learning and imitation learning methods, and where world model training typically relies on interactive trajectories.

World Models (Ha et al., 2018). Ha and Schmidhuber demonstrated that compact forward dynamics models can enable learning effective control policies largely through imagined rollouts. Their pipeline compresses pixel observations into latent states using a Variational Autoencoder (VAE) (Kingma et al., 2013) and learns action-conditioned dynamics in this latent space via a MDN-RNN (Graves, 2013), enabling the agent to train a controller using imagined trajectories rather than direct environment interaction. The key achievement is showing that learned latent dynamics can be sufficient for solving visual control tasks and support transfer to the real simulator. However, the approach assumes access to ground-truth actions and additional environment signals e.g. termination and reward, and the learned representation is shaped by reconstruction-style objectives that may encode nuisance details.

Dreamer family (Hafner et al., 2023). Dreamer extends this paradigm into a high-performance model-based RL framework that learns a latent world model and improves behavior through imagined trajectories. DreamerV3 demonstrates robust learning across a wide range of domains and tasks without extensive task-specific hyperparameter tuning, establishing that latent world models can scale to diverse and challenging environments. It uses a VAE encoder and an RSSM (Hafner et al., 2018) latent dynamics predictor. Relative to our setting, Dreamer remains fundamentally interactive: training assumes action- and reward-labeled trajectories and yields task-specific policies through actor–critic learning rather than enabling explicit goal-conditioned test-time replanning under a reusable dynamics model.

Transformer-based world models (TransDreamer) (Chen et al., 2022). TransDreamer replaces recurrent dynamics with transformer-based sequence modeling to better capture long-horizon dependencies. Empirically, it improves prediction quality and downstream performance on tasks requiring longer memory. This motivates transformer predictors for dynamics, but it does not address our supervision regime: it still relies on action- and reward-labeled interaction data.

3.2 Latent Action Policy Learning

A complementary research direction occupies the offline, observation-centric policy-learning regime. Rather than relying on action-labeled trajectories, these methods infer latent actions from observation transitions and learn policies in the resulting latent action space.

ILPO (Edwards et al., 2018). ILPO learns a latent policy from state-only demonstrations and then uses limited interaction to align latent actions with executable actions. These works establish that inverse dynamics and latent actions can bridge observation-only data to control, but they are not designed around embedding-space goal-conditioned planning and typically require additional interaction or alignment steps.

LAPO (Schmidt et al., 2023). LAPO jointly trains an inverse dynamics model (IDM) and a forward dynamics model (FDM) on observation-only transitions. The IDM infers a latent action from consecutive observations, while the FDM predicts the next observation conditioned on the current observation and the inferred latent action. A vector-quantized bottleneck constrains the latent action to a compact discrete codebook, encouraging reuse and discouraging trivial encodings. LAPO demonstrates that meaningful action-like abstractions can be recovered without action labels and used for downstream control, including adaptation to real action spaces with minimal supervision. Relative to our objective, LAPO is primarily evaluated as a policy-learning pipeline via behavior cloning, whereas our focus is explicit planning and optimization at decision time under a learned dynamics model.

UniVLA (Bu et al., 2025). UniVLA addresses scalability and embodiment heterogeneity by learning discrete latent action tokens from observation pairs in a pretrained visual feature space, via an IDM and FDM with a VQ bottleneck. After labeling the data with the frozen

IDM, a transformer policy is trained to predict these tokens. Notably, it incorporates language conditioning and a two-stage decoupling procedure to separate task-irrelevant dynamics e.g., non-ego object movement, from task-centric actions. A key achievement is demonstrating that latent action learning in pretrained representations, notably DINOv2 features, can support high-performing robot policies and efficient downstream adaptation via lightweight decoders into embodiment-specific action spaces. For our purposes, UniVLA motivates latent action learning in pretrained embedding spaces, but it remains primarily a policy-learning approach: new tasks typically require retraining or conditioning the policy rather than replanning through a reusable learned dynamics model.

3.3 Planning with Model-based RL

We now turn to approaches that explicitly employ learned dynamics at decision time to plan or optimize action sequences for control. Planning-based control can adapt to new goals or cost functions without retraining a policy, aligning with the goal-conditioned setting studied in this thesis.

TD-MPC2 (Hansen et al., 2023). TD-MPC2 is a planning-centric model-based RL method that learns a latent dynamics predictor without pixel reconstruction and performs model-predictive control using sampling-based trajectory optimization. It achieves strong performance across diverse continuous-control tasks and provides evidence that effective control can be achieved by planning directly in a learned latent space. However, TD-MPC2 relies on action-labeled interaction data and reward/value supervision, preventing direct training from passive, action-free video.

DINO-WM (Zhou et al., 2024). DINO-WM aims to enable task-agnostic goal-reaching via test-time planning without explicit rewards or expert demonstrations. It learns a dynamics predictor directly in a frozen pretrained visual representation space (DINOv2 features), avoiding pixel reconstruction and leveraging semantically structured embeddings. At test time, DINO-WM performs goal-conditioned MPC using CEM by selecting action sequences whose predicted future features best match a goal feature representation. DINO-WM demonstrates strong zero-shot planning capabilities on multiple benchmarks. The key limitation for our thesis is that the learned dynamics remain conditioned on ground-truth executed actions during training.

MuZero (Schrittwieser et al., 2020). MuZero learns a planning-oriented latent model and performs control via Monte Carlo tree search (MCTS), predicting only the quantities needed for search (e.g., reward and value) rather than reconstructing observations. It achieves strong performance in both discrete games and visually complex environments, demonstrating that coupling representation learning to planning can be highly effective. However, MuZero remains tied to action-labeled interactive RL and reward-driven objectives. While its reliance on MCTS is an interesting alternative to sampling-based methods, it can come with substantial computational overhead.

3.4 Planning in Latent Action Space

Latent action models are often studied for policy learning, but there is also work showing that latent actions can be used for inference-time optimization and planning.

Efficient Planning in a Compact Latent Action Space (Jiang et al., 2022). Jiang et al. study how planning can be made more tractable by optimizing over a compact latent action space rather than the native action space. They learn low-dimensional latent action codes using a state-conditioned VQ-VAE, which serves as a dynamics model for predicting long-horizon trajectories given an initial state and a latent action code. The key achievement is demonstrating that an action bottleneck can improve planning efficiency and regularize action sequences, especially in settings where raw action optimization is difficult. This supports the general assumption that planning can benefit from structured latent action representations, while their VQ-VAE approach and reliance on action- and reward-labeled trajectories differ from our method.

AdaWorld (Gao et al., 2025). AdaWorld proposes action-aware pretraining from unlabeled video by learning latent actions from visual transitions and using them to enable controllable prediction and planning. It demonstrates that continuous latent actions learned from observation pairs via β-VAE (Higgins et al., 2017) can support goal-based MPC with CEM optimization (Rubinstein, 1999) and can be aligned to real actions with limited labeled interaction data. AdaWorld is particularly relevant because it evaluates planning behavior under learned latent actions, but it differs from our focus in that its training and planning are centered on generative prediction objectives, a decoupled latent-action module and a continuous information bottleneck, whereas we aim to learn controllable dynamics directly in a frozen embedding space and couple inverse and forward models under a joint predictive objective with discrete latent actions.

3.5 Joint-Embedding Predictive World Models

To provide a more complete picture of recent progress toward scalable representation-space dynamics, we include joint-embedding predictive models explicitly specialized for temporal prediction in latent space.

V-JEPA 2-AC (Assran et al., 2025). V-JEPA 2 performs large-scale action-free video pretraining using a joint-embedding predictive objective, producing representations optimized for capturing object-level structure and temporal dynamics without pixel reconstruction. In a second stage, V-JEPA 2-AC freezes the encoder and trains an action-conditioned predictor for planning on robot data with an embodiment-specific action interface. At test time, it performs goal-conditioned MPC using CEM in representation space and demonstrates strong planning performance, including in real-robot settings. While this line of work strongly supports the frozen-encoder, learned-predictor and test time optimization, it still requires an action interface at the predictor level, motivating the question of whether learned latent actions can replace action labels while retaining planning capability.

3.6 Thesis Positioning

The reviewed literature spans three complementary approaches to learning models for control. Interactive latent world-model methods such as World Models (Ha et al., 2018) and DreamerV3 (Hafner et al., 2023) learn action-conditioned dynamics from interaction trajectories and learn behavior through imagined rollouts, achieving strong performance but relying on action and reward supervision. Planning-centric methods such as TD-MPC2 (Hansen et al., 2023), MuZero (Schrittwieser et al., 2020), and embedding-space planners such as DINO-WM (Zhou et al., 2024) explicitly optimize or search over action sequences at decision time and often avoid pixel reconstruction by operating in latent spaces, but they remain tied to ground-truth action signals for learning controllable dynamics. In parallel, observation-centric latent-action approaches ranging from ILPO to LAPO and UniVLA (Edwards et al., 2018; Schmidt et al., 2023; Bu et al., 2025) infer action-like representations from transitions and enable policy learning with reduced action supervision, but are most often evaluated in policy-learning regimes rather than in goal-conditioned test-time planning with reusable dynamics.

As a result, the literature provides limited evidence on whether the goal-conditioned test-time planning benefits of latent space world models can be retained when action conditioning is replaced by learned latent actions inferred from video transitions. This thesis is situated at the intersection of latent space world models and latent-action learning (Table 1). It builds on the DINO-WM paradigm of learning dynamics in a frozen DINOv2 representation space and performing goal-conditioned optimization via CEM, while adopting joint inverse/forward modeling characteristic of latent-action approaches and drawing on latent-action planning insights that motivate an action bottleneck for optimization (Schmidt et al., 2023; Bu et al., 2025; Gao et al., 2025). Concretely, we study a DINO-WM variant whose dynamics are conditioned on latent actions. Inspired by UniVLA, LAPO, and AdaWorld (Bu et al., 2025; Schmidt et al., 2023; Gao et al., 2025), these latent actions are inferred by an inverse dynamics model rather than taken from ground-truth executed actions.

4 DINO World Model with Inverse Dynamics

4.1 Overview

Following the POMDP formulation introduced in Section 2.1, we consider a vision-based interaction setting in which the environment generates an observation stream $o_t = x_t \in \mathcal{O}$. In contrast to the standard control setting, we assume access only to observations and do not use ground-truth action labels. Instead, we introduce a learned discrete latent action variable $u_t \in U$ that represents the unobserved transition between consecutive observations. We parameterize the latent action space as $U = \{1, \ldots, K\}$.

With the proposed method, we aim to learn a task-agnostic, latent-action-conditioned world model from offline, image-only trajectories collected in the PushT environment (Chi et al., 2023). The learned model is used at test time to solve visual, goal-directed tasks via trajectory optimization in the discrete latent action space. Planning is performed by rolling out sequences of latent actions in the learned dynamics model such that the final predicted latent representation minimizes the distance to a provided goal embedding. In contrast to prior work (Hafner et al., 2023; Hansen et al., 2023; Zhou et al., 2024), we do not model rewards and do not rely on ground-truth action labels, enabling fully action-label-free pretraining and an embodiment-agnostic latent action space.

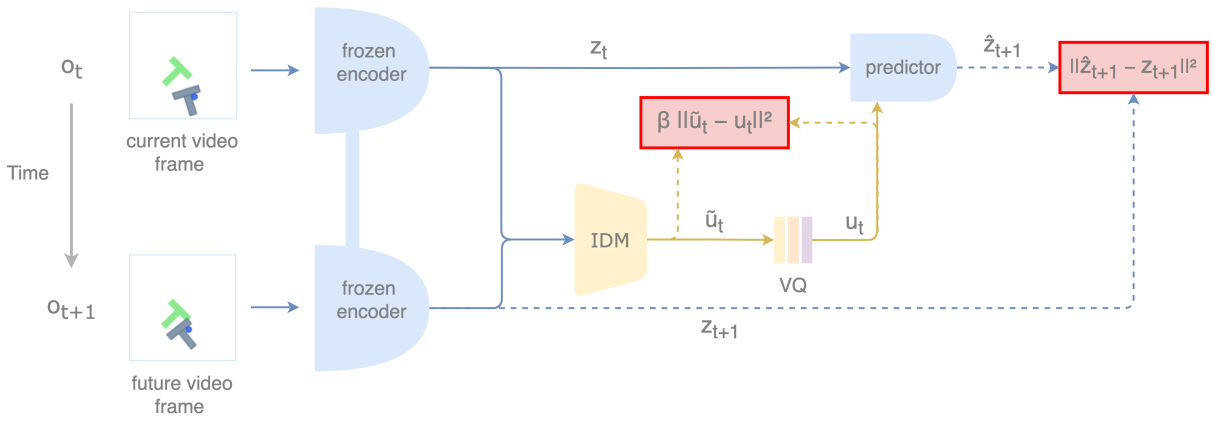

Figure 1: Overview of the proposed architecture. A frozen encoder maps observations to latent representations. The inverse dynamics model infers a latent action, which is subsequently quantized by a vector-quantization bottleneck. The forward dynamics model then predicts the next latent representation. The model is trained using a prediction loss for the dynamics model, while the inverse dynamics model is optimized using the same prediction loss augmented with an additional VQ commitment loss. Example input images from (Zhou et al., 2024) while the figure is inspired by (Assran et al., 2025).

At each time step t, the world model consists of the following components:

$$\begin{aligned}
\textbf{Encoder:} \quad & z_t = \mathrm{enc}_\psi(o_t), \\
\textbf{Inverse dynamics model:} \quad & \tilde{u}_t = q_\omega(z_t, z_{t+1}), \\
\textbf{Vector quantization:} \quad & u_t = \mathrm{VQ}(\tilde{u}_t), \\
\textbf{Forward dynamics model:} \quad & \hat{z}_{t+1} = f_\theta(z_t, u_t).
\end{aligned}$$

The encoder maps image observations o_t and o_{t+1} to latent representations z_t and $z_{t+1} \in \mathcal{Z}$. The inverse dynamics model infers a continuous latent action $\tilde{u}_t$ from the latent state transition z_t, z_{t+1}. This latent action is then quantized to the nearest codebook entry, resulting in the discrete codebook vector u_t. The forward dynamics model predicts the next latent representation $\hat{z}_{t+1}$ conditioned on z_t and u_t, operating entirely in latent space.

Following DINO-WM the forward dynamics model is trained solely using a mean squared error (MSE) prediction loss, while the inverse dynamics model is optimized using the same prediction loss augmented with an additional VQ commitment loss. This joint training procedure ensures that the inferred latent actions are optimized to support accurate forward prediction in latent space. The following sections describe the individual components in detail, beginning with the encoder, followed by the inverse dynamics module, the forward dynamics model, the training procedure, the visual planning method, and the latent action decoder.

4.2 Encoder

Our objective is to learn a task- and environment-agnostic world model. Accordingly, the role of the encoder is to provide rich, general-purpose spatial features that support downstream inverse dynamics learning and latent-space planning across diverse environments.

Rather than training the visual encoder exclusively on the target task or environment (Hansen et al., 2023; Hafner et al., 2020), we rely on a pretrained model that captures general visual structure. This design choice is motivated both by computational constraints and by the observation that visual perception is largely task-independent and benefits substantially from large-scale pretraining. Consistent with the DINO-WM framework (Zhou et al., 2024), we adopt a pretrained ViT-S/14 DINOv2 model (Oquab et al., 2023) as the perceptual backbone. DINOv2 has demonstrated strong spatial representations across a variety of dense visual tasks, including object detection, semantic segmentation, and depth estimation, while comprising only 21 million parameters, making it well suited to our setting.

At each time step t, DINOv2 processes a single image observation o_t and maps it to a set of patch embeddings $z_t \in \mathbb{R}^{256 \times 384}$. These embeddings correspond to a 16×16 grid of non-overlapping 14×14 patches, each with an embedding dimension of 384. We retain the full set of patch embeddings as the latent state z_t, which is consumed directly by downstream transformer-based components without spatial pooling. The model is pretrained at a resolution of 224×224, which we retain throughout all experiments, with images normalized using the standard DINOv2 preprocessing. The encoder is kept frozen during both training and planning, therefore all

temporal structure and action-related abstraction must be captured by the inverse dynamics and forward dynamics models.

4.3 Inverse Dynamics Model

For the inverse dynamics model (IDM), we reuse the released implementation of AdaWorld's latent action autoencoder (Gao et al., 2025), which is based on the spatiotemporal Transformer (SP-Transformer) architecture introduced in Genie (Bruce et al., 2025). This factorized spatiotemporal attention design balances representational capacity with computational and memory constraints. Compared to the original formulation, several modifications are introduced to adapt the model to our architecture and training setup.

First, instead of operating on raw image frames, the IDM takes patch-level visual token representations produced by the encoder as input. Consequently, the patchification and patch embedding layers of the original model are removed. Second, while the original AdaWorld model employs a latent action decoder to reconstruct the subsequent frame in pixel space, the IDM is used as an encoder-only model. Since the downstream forward dynamics model assumes the role of the decoder, the latent action decoder is disabled entirely. Third, no posterior distribution over latent actions is parameterized, and no sampling or KL regularization is performed; instead, the deterministic output of the SP-Transformer is used directly as the latent action representation.

The SP-Transformer used in the IDM consists of multiple stacked spatiotemporal blocks, each following the standard Transformer structure of attention layers followed by a feed-forward network. Within each block, attention is factorized into spatial and temporal components. Spatial self-attention is applied independently at each time step and operates over all tokens within a frame, attending to the $1 + H \times W$ tokens corresponding to the action token and the spatial patch tokens. Temporal self-attention is applied independently at each token index and aggregates information across the temporal dimension by attending to the tokens corresponding to the same spatial position across time. Temporal attention is implemented causally using a causal mask, ensuring that representations at time step t do not attend to future observations. All attention layers use multi-head self-attention, no dropout is applied, and each block includes a position-wise feed-forward network as in standard Transformer architectures.

The goal of the IDM is to provide a consistent and useful abstract code for transitions between consecutive latent states, ideally serving as a signal for the forward dynamics model and rendering the prediction of the next latent representation unimodal in the PushT environment. At each time step t, the IDM receives the latent representations z_t and z_{t+1}, corresponding to a temporal window of two consecutive frames, represented as sequences of patch-level tokens. For each frame, a learnable action token is prepended as the first token to the corresponding patch tokens, forming an augmented token set consisting of one action token and multiple spatial tokens per frame.

The resulting spatiotemporal token sequence is processed by the SP-Transformer. Through attention, the learnable action tokens attend to both spatial and temporal context, progressively accumulating motion-relevant features. After encoding, all patch tokens are discarded, and the

encoded action token corresponding to time step $t+1$ is selected as the latent action representation $\tilde{u}_t \in \mathbb{R}^{384}$, capturing the transformation from z_t to z_{t+1}.

Since the dimensionality of the latent action $\tilde{u}_t$ matches that of the latent representation z_{t+1}, the inverse dynamics encoder could trivially copy z_{t+1} into $\tilde{u}_t$. This would allow the downstream forward dynamics model to approximate an identity mapping, yielding a degenerate yet optimal prediction. To prevent this shortcut solution, the latent action representation is constrained to a lower-dimensional space and regularized using vector quantization.

Concretely, the continuous latent action vector $\tilde{u}_t$ is down projected by a learnable liner layer and quantized by assigning it to the nearest entry in a learned codebook of K discrete latent action prototypes. The quantized latent action is used in the forward pass via a straight-through estimator, enabling gradient propagation through the IDM while enforcing a discrete bottleneck. The codebook is updated using an exponential moving average scheme that tracks assignment frequencies and running averages of IDMs outputs. This approach stabilizes training and avoids codebook collapse without relying on direct gradient updates. A commitment loss further encourages IDM outputs to remain close to their selected codebook entries, promoting stable and meaningful discretization. Overall, this discrete bottleneck enforces a compact latent action space and ensures that the forward dynamics model must learn non-trivial state transitions conditioned on $\tilde{u}_t$, rather than exploiting direct information copying. For the ablation study, we include a model variant that discards VQ quantization and directly uses the latent action for downstream components. The inverse dynamics model is trained jointly with the forward dynamics model and receives gradients from the latent dynamics prediction loss.

4.4 Forward Dynamics Model

We reuse the predictor introduced in DINO-WM (Zhou et al., 2024) as our forward dynamics model. It is based on a Vision Transformer (ViT)–style architecture (Dosovitskiy et al., 2020), which is well suited for modeling sequences of latent tokens. Since the inputs to our model are already tokenized latent representations, the patch tokenization stage used in standard ViT architectures is omitted. Consequently, the model operates directly on sequences of latent tokens using masked self-attention, resulting in a decoder-style transformer.

The predictor is implemented as a multi-layer Transformer network using multi-head self-attention and feed-forward sublayers. We employ a standard pre-layer normalization design with residual connections, and apply dropout within the Transformer sublayers to improve generalization.

Temporal dependencies are captured through a frame-level self-attention mask to ensure that representations at time step t do not attend to future observations, while attention within the same time step remains fully connected across tokens. This enforces autoregressive prediction at the frame level, treating the patch tokens of an observation as a single unit. This design differs from token-level autoregressive approaches such as IRIS (Micheli et al., 2022), which additionally imposes an ordering within the patch tokens of the current frame. By modeling temporal dependencies across complete observations rather than individual tokens, this approach better captures global structure and temporal dynamics (Zhou et al., 2024).

In our setting, we retain the architectural design of the predictor but modify its inputs and training signals. Specifically, we do not use proprioceptive information. To preserve architectural compatibility with the DINO-WM implementation, we keep the proprioceptive input interface but set the corresponding tokens to zero for all samples and time steps, and remove the associated proprioceptive loss term. As a result, the latent dynamics objective is applied exclusively to the visual latent tokens, and the model learns purely visual latent dynamics. We argue that this step is advantageous for training on raw video data, where neither proprioceptive signals nor action labels are available.

Furthermore, instead of conditioning the predictor on ground-truth actions, we condition it on latent actions inferred by the inverse dynamics model. Unlike the original DINO-WM formulation, which predicts future latent states conditioned on a history of multiple past frames, we use a history length of one and predict the next latent state from the current frame only. This design choice is motivated by two considerations. First, the inverse dynamics model infers latent actions from pairs of consecutive frames, and using a history length of one allows both models to operate on the same temporal input structure and data pipeline. Second, in environments with relatively smooth and predictable dynamics, longer observation histories may reduce reliance on action conditioning by enabling temporal extrapolation from visual information alone. To mitigate this effect and encourage the forward dynamics model to make use of the inferred latent action, we restrict the history length to one, which also significantly reduces computation due to sequence-length-dependent attention. Due to time constraints, we do not include a systematic ablation study over different history lengths in this work.

At each time step t, the forward dynamics model takes the latent representation z_t. Each visual patch token is augmented by concatenating the tiled latent action u_t along the feature dimension, yielding an augmented representation z_t^u that jointly encodes visual and latent action information. The model then predicts the next latent visual representation z_{t+1}^u. The action features are used solely for conditioning and are explicitly excluded from the prediction loss. Consequently, the model does not learn to predict latent actions. Training is performed using a mean squared error (MSE) loss between the predicted and target latent visual representations, computed over all visual patch tokens and time steps in the prediction window.

4.5 Training Procedure

Training is conducted in latent space using patch-level representations z_t produced by the encoder, which remains frozen throughout all experiments.The decoder is omitted due to compute constraints, since it does not affect the model and would be used only for visualization; moreover, we do not use a pixel reconstruction loss. The system is trained on transition windows of length two, i.e., each training sample consists of consecutive latent states (z_t, z_{t+1}). Training data are generated by temporally subsampling each trajectory and then extracting all valid windows with stride one. Each dataset index corresponds to exactly one window. The DataLoader is instantiated with shuffling enabled, so mini-batches are formed from randomly permuted transition windows.

The IDM and the forward dynamics model are optimized jointly end-to-end. The action and

proprioception encoders used in the original DINO-WM formulation are disabled and not used. Let $\hat{z}_{t+1}$ denote the predicted next latent representation produced by the forward dynamics model. The forward dynamics model is optimized using

$$\mathcal{L}_{\text{pred}} = \|\hat{z}_{t+1} - z_{t+1}\|_2^2 \,,$$

where the norm is taken over all token and feature dimensions and averaged over the batch. The IDM is additionally regularized using the commitment loss

$$\mathcal{L}_{\text{commit}} = \|\text{sg}(u_t) - \tilde{u}_t\|_2^2 \,,$$

where $\tilde{u}_t$ denotes the continuous latent action produced by the IDM prior to quantization, u_t denotes the corresponding quantized latent action e.g. the nearest codebook vector, and $\text{sg}(\cdot)$ denotes the stop-gradient operator. The total training objective is defined as

$$\mathcal{L} = \mathcal{L}_{\text{pred}} + \lambda \, \mathcal{L}_{\text{commit}},$$

with a fixed weighting coefficient λ. During training, gradients from $\mathcal{L}_{\text{pred}}$ propagate through the forward dynamics model into the IDM. Trainable parameters are optimized using two separate optimizers targeting different model components. The forward dynamics model and the inverse dynamics model are trained using L_{pred}. The latent-action pathway, including the inverse dynamics model and all vector-quantization parameters, is trained using L_{commit}. The codebook is updated via exponential moving averages and is not optimized via gradient descent.

4.6 Visual Planning

We retained the original planning formulation (Zhou et al., 2024) but modified its implementation by using an optimizer tailored to discrete action spaces, since planning is performed over the latent action codebook rather than a continuous Gaussian distribution. Across all experiments, we use CEM for planning, motivated by the strong results reported for PushT with this combination in the original setup.

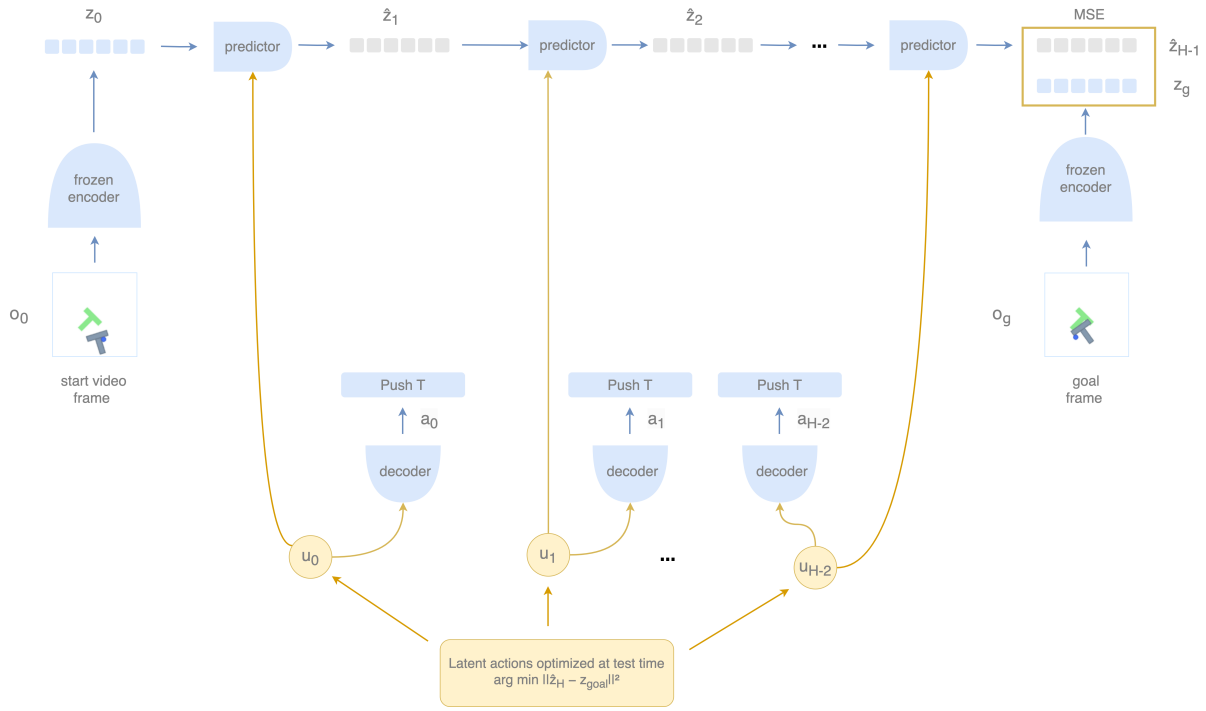

Figure 2: Overview of the visual planning procedure. A frozen visual encoder maps observations to latent representations. Candidate sequences of discrete latent actions are optimized by rolling them out in the learned forward dynamics model and selecting sequences whose predicted terminal latent states best match the goal representation. Example input images and inspiration from (Zhou et al., 2024).

For the ground-truth action baseline, we use the original planning implementation without modification. Given a current observation o_0 and a goal observation o_g, the planner encodes both using the frozen encoder to obtain $z_0 = \text{enc}_\psi(o_0)$ and $z_g = \text{enc}_\psi(o_g)$. It then optimizes an action sequence over a planning horizon H by evaluating candidate rollouts with the forward dynamics model. For each candidate plan, the forward dynamics model is rolled out from z_0 while conditioning on the latent-action sequence, producing a terminal prediction $\hat{z}_H(u)$. The planning objective is defined as the terminal latent distance to the goal,

$$C = \|\hat{z}_H(u) - z_g\|_2^2,$$

where the norm is taken over all token and feature dimensions.

Since control is performed over the discrete VQ codebook, we use a discrete Cross-Entropy Method (CEM) variant for planning in the latent-action setting (De Boer et al., 2005). Let U denote the discrete VQ codebook, and denote a plan by a latent-action sequence $u = (u_0, \ldots, u_{H-1})$ with

$u_t \in U$. CEM runs for I optimization iterations. At iteration $i \in \{1, \ldots, I\}$, a population of N candidate plans $\{u^{(n)}\}_{n=1}^{N}$ is sampled from a factorized categorical distribution by drawing indices $j_t^{(n)} \sim \mathrm{Cat}(\pi_t)$ for $t = 0, \ldots, H-1$ and selecting the corresponding codebook entries. Each candidate is evaluated by rolling out the forward dynamics model and computing its cost $C^{(n)}$. An elite set containing the M lowest-cost candidates is selected, and the categorical parameters are updated by maximum-likelihood estimation under the elite set, using pseudocount smoothing and a momentum update to stabilize convergence. This sample–evaluate–select–update loop is repeated for I iterations, after which the final plan is obtained.

4.7 Latent Action Decoder

To execute the optimized latent-action plan in the environment, the learned latent actions must be mapped to the environment-specific action space. We therefore train a lightweight MLP decoder to perform this mapping. The latent action decoder is trained separately from the world model and planning components using a set of labeled transition pairs.

Given a labeled transition, the decoder receives the latent action $u_t \in U$ and the corresponding tokenized latent state representation z_t. We apply attention pooling over the patch-token sequence to obtain a pooled representation, and concatenate it with an embedding of the latent action to form the decoder input. The MLP then predicts the continuous ground-truth environment action $a_t \in \mathbb{R}^{10}$. The decoder is optimized using a supervised mean squared error objective on the labeled set.

5 Experimental Setup

5.1 Overview

This chapter describes the experimental design and evaluation strategy used to investigate the central research question introduced in Chapter 1. Due to time and computational constraints, we evaluate three fixed model variants of the DINO World Model on the PushT benchmark. These variants consist of an action-label-conditioned baseline (Baseline), a latent action variant without vector quantization (LA), and a variant with discrete latent actions (LA-VQ). Unless stated otherwise, architectural and optimization hyperparameters are adopted from prior work (Zhou et al., 2024; Gao et al., 2025; Schmidt et al., 2023).

The primary goal of the experiments is to assess whether a latent action formulation can achieve planning performance comparable to that of the action-conditioned baseline, and to characterize the qualitative and quantitative differences between the three variants.

The chapter proceeds as follows. We first introduce the expected failure modes of latent action world models and describe metrics to evaluate them. We then present the environment and dataset, implementation details, and evaluation protocol, followed by the experiments.

5.2 Diagnostics of Failure Modes

Beyond task-level performance, we propose a set of diagnostics to characterize potential failure modes of our latent action variants. These measures are evaluated after training and are intended to support qualitative and quantitative interpretation of the experimental results.

Information-Bypass Collapse (Shortcut Learning). In the information-bypass collapse mode, the inverse dynamics model encodes excessive information about the next latent state z_{t+1} into the latent action $\tilde{u}_t$. As a result, the forward dynamics model $f_\theta(z_t, \tilde{u}_t)$ can predict z_{t+1} primarily by decoding $\tilde{u}_t$, rather than learning a meaningful state-transition model conditioned on the latent action variable (Schmidt et al., 2023). In this case, the latent actions no longer represent controllable interventions, making planning ill-posed. To measure this failure, we apply a counterfactual test[1] . For two samples i, j drawn from a batch, we combine $z_t^{(i)}$ with $\tilde{u}_t^{(j)}$ and compute $\hat{z}^{(i,j)} = f_\theta(z_t^{(i)}, \tilde{u}_t^{(j)})$, then evaluate whether $\hat{z}^{(i,j)}$ is closer to $z_{t+1}^{(j)}$ than to $z_{t+1}^{(i)}$. This defines the score

$$s = \mathbb{P}\Big(\|\hat{z}^{(i,j)} - z_{t+1}^{(j)}\| < \|\hat{z}^{(i,j)} - z_{t+1}^{(i)}\|\Big)$$

We therefore estimating how strongly $\tilde{u}$ determines the prediction.

Action-Ignoring Collapse. In the action-ignoring collapse mode, the forward dynamics model learns to predict $\hat{z}_{t+1} \approx f(z_t)$ largely independent of the latent action u_t, which can occur when the environment dynamics are smooth or the dataset is dominated by highly predictable motions. In this case, the IDM and vector quantization stages may still appear numerically well-behaved, yet the action signal is effectively unused. As a result planning over u_t becomes meaningless. Many distinct action sequences yield nearly identical rollouts, eliminating controllability. To detect this behavior, we apply an action-shuffle test[2]. We first compute the baseline prediction error

$$\mathrm{MSE}_{\mathrm{base}} = \|f(z_t, u_t) - z_{t+1}\|^2,$$

and then recompute the error after shuffling actions across the batch while keeping states fixed,

$$\mathrm{MSE}_{\mathrm{shuf}} = \|f(z_t, \mathrm{shuffle}(u_t)) - z_{t+1}\|^2.$$

From this we define the action-dependence score

$$\Delta_{\mathrm{shuf}} = \frac{\mathrm{MSE}_{\mathrm{shuf}} - \mathrm{MSE}_{\mathrm{base}}}{\mathrm{MSE}_{\mathrm{base}} + \varepsilon}.$$

If the model meaningfully uses u_t, shuffling should substantially degrade prediction quality.

Latent-Action Collapse to a Constant Code. In the latent-action collapse mode, the inverse dynamics model produces nearly identical latent actions $\tilde{u}_t$ for most transitions, causing the vector quantizer to assign the majority of samples to a single codebook entry or a small subset of entries. This failure can occur even when the codebook itself does not numerically

[1]This diagnostic test was proposed with assistance from ChatGPT (GPT-5.2, OpenAI).
[2]This diagnostic test was proposed with assistance from ChatGPT (GPT-5.2, OpenAI).

collapse and maintains distinct embeddings. The resulting behavior is harmful because planning over action sequences becomes trivial and ineffective. The model effectively loses controllable degrees of freedom (Oord et al., 2017). To detect this collapse, we monitor code usage statistics by computing the empirical assignment distribution $p(k)$ over codebook entries. From this, we compute the codebook perplexity

$$\mathrm{PPL} = \exp\left(-\sum_{k=1}^{K} p(k)\, \log\big(p(k) + \varepsilon\big) \right),$$

and normalize it as

$$\mathrm{PPL}_{\mathrm{norm}} = \frac{\mathrm{PPL}}{K}.$$

In a healthy model, a substantial fraction of the codebook should be utilized.

Codebook Collapse and Dead-Code Pathologies. In this collapse mode, which is specific to vector-quantized models, the codebook itself degenerates despite the use of EMA updates. Typical manifestations include dead codes that are never selected, code vector crowding where multiple embeddings converge to similar directions and thereby reduce effective capacity, and high assignment churn in which code usage changes erratically over time, often caused by a mis-scaled commitment loss or learning-rate mismatch (Oord et al., 2017). We measure the dead-code rate by counting the fraction of codebook entries that are never selected,

$$\mathrm{dead_rate} = 1 - \frac{\#\{k : \mathrm{count}(k) > 0\}}{K}.$$

Multi-step rollout drift. This failure mode arises when the model is trained using a one-step prediction loss, while planning relies on multi-step rollouts. Small systematic prediction biases that are negligible at one step can accumulate over time, leading the planner to exploit model inaccuracies and produce trajectories that appear plausible in latent space but are incorrect in the environment (Talvitie, 2016). As a diagnostic, we compare multi-step rollout error on held-out trajectories under ground-truth actions and inferred latent codes. A substantially larger error under latent-action rollouts indicates instability in the learned latent dynamics.

Execution failure due to latent-action decoding. In this failure mode, planning yields sequences of discrete latent actions, but decoding them into executable environment actions is insufficiently accurate or stable. To diagnose this issue, we evaluate the latent-action decoder in isolation by computing the root mean squared error (RMSE) on the validation dataset and compare it against the mean action predictor baseline.

5.3 Environment and Dataset

Environment. All experiments are conducted in the PushT environment used in the DINO-WM paper (Zhou et al., 2024), using the original setup and observation space (RGB images at 224×224 resolution). PushT (Chi et al., 2023) features a pusher interacting with a T-shaped block. From randomized initial states, the objective is to move both the agent and the T-block

to a feasible target configuration within 25 steps. In the DINO-WM setting, the fixed green "T" serves solely as a visual anchor and does not represent the target, unlike in the original implementation. An episode is considered successful if the agent and target positions are within a Euclidean distance of 20 units and the orientation error is less than $\pi/9$, accounting for angular wrap-around. The environment uses a continuous two-dimensional action space, where the agent directly controls planar motion. Overall, the task emphasizes contact-rich visuomotor control and requires precise object manipulation.

Due to time constraints, we focus on environments that are already implemented and well integrated within the DINO-WM framework. We select PushT as our benchmark among those environments. Compared to navigation tasks such as PointMaze and Wall, PushT introduces contact-rich object interaction, requiring reasoning about contact-induced object translation and rotation while remaining a low-dimensional, planar task with a simple action space. In contrast, rope and granular manipulation tasks involve three-dimensional deformable dynamics and substantially more complex action–effect relationships, where performance is dominated by modeling and control challenges rather than the properties of the latent action representation itself.

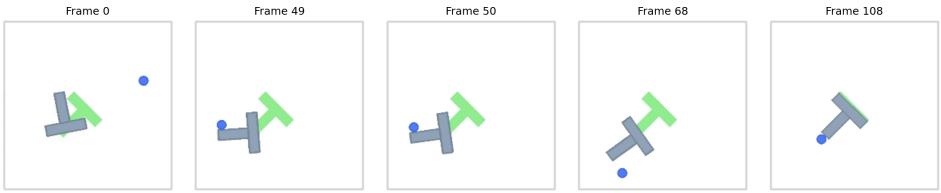

Figure 3: Example observation from a training trajectory illustrating the PushT environment. Sampled fron the PushT noise dataset (Zhou et al., 2024).

Dataset. We use the PushT-noise dataset introduced in the DINO-WM paper (Zhou et al., 2024). It consists of 18,500 manipulation trajectories generated by replaying the originally released expert demonstrations with varying levels of injected noise. It includes continuous action labels, velocities, states, and per-trajectory MP4 videos. Actions are defined as relative, between-frame displacements in the environment plane, represented as pixel-space offsets $(\Delta x, \Delta y)$ on a 512×512 grid.

Due to computational constraints, we train on a uniformly random subsample of 10,000 trajectories (denoted `push_t_noise_10k`). Based on the scaling ablation reported in the original paper (Zhou et al., 2024), this subset provides a good trade-off between performance and computational cost. We additionally provide a minimal 1,000-trajectory subset (`push_t_noise_1k`) for low-resource experiments. Subsampling is performed at the trajectory level to avoid data leakage across splits. Since the original validation split contains only 20 trajectories, we partition the training split into 9000 training and 1000 validation trajectories using a 0.9/0.1 ratio, ensuring strict separation at the trajectory level. All datasets retain the original file structure, and we provide dataset access at https://github.com/pythonpedro99/dino_wm_latent_actions.

5.4 Implementation

This section documents the implementation details required to reproduce all experiments reported in this thesis. We explicitly specify the software and hardware environment, model architecture, optimization procedure, planning algorithm, and the configuration of all evaluated model variants. All variants are implemented within a single codebase and differ only through a small set of gating parameters and variant-specific hyperparameters. The complete implementation, including instructions for reproducing the reported results, environment configuration files, variant-specific configurations, and execution commands, is publicly available at: https://github.com/pythonpedro99/dino_wm_latent_actions. All results reported in this thesis correspond to commit `10133ec`. All model variants are implemented in PyTorch 2.3 and trained with CUDA 12.1 on a single NVIDIA L4 GPU using the Python 3 Google Compute Engine Backend (GPU) in Colab. All models share a common world-model backbone consisting of a ViT-S/14 DINOv2 encoder (Oquab et al., 2023) and a PyTorch-based Vision Transformer forward dynamics model (Zhou et al., 2024). The architecture of the forward dynamics model is summarized in Table 2, while details of the encoder architecture are provided in the original DINOv2 publication.

Table 2: Vision Transformer predictor architecture.

Parameter	**Value**
Transformer depth	6
Number of attention heads	16
MLP dimension	2048
Dropout	0.1
Embedding dropout	0.0

Random Seed Handling. All experiments are executed with fixed random seeds to ensure reproducibility across all algorithmic sources of randomness. A global seeding routine initializes the random number generators of Python, NumPy, PyTorch (CPU), and CUDA at the start of training and planning. Training uses a configurable seed that is applied prior to dataset creation, data loader initialization, and model parameter initialization. Training data are shuffled using a seeded DataLoader generator, yielding a reproducible sample order for a fixed seed, while validation data are iterated in a deterministic order. Open-loop rollouts and planning experiments similarly reset the relevant random number generators before sampling trajectories, horizons, or environments. Planning evaluations derive deterministic per-run seeds from a fixed base seed, ensuring reproducible rollouts across evaluation runs.

Several components are stochastic by design, including trajectory segment sampling, random action goal generation, and rollout sampling. These processes are fully controlled by the global seeding mechanism and are therefore deterministic under fixed seeds. Complete bit-level determinism is not guaranteed: GPU execution introduces inherent floating-point nondeterminism, and data loader worker processes do not explicitly reinitialize local RNG states. Consequently, repeated runs with identical seeds are expected to be reproducible up to minor numerical differences arising from parallel and GPU-accelerated computation.

Training Setup. Training is performed using separate AdamW optimizers for the predictor, action encoder, and latent-action model, where applicable. Optimizer types, learning rates, and training schedules are fixed across variants unless explicitly overridden. All experiments use a fixed batch size and number of training epochs. Checkpointing is performed at regular intervals, and the final checkpoint is used for evaluation unless stated otherwise. Table 3 reports the training hyperparameters shared across all variants. Optimizer parameters correspond to default values and are included for completeness.

Table 3: Training and model configuration parameters used in all experiments.

Parameter	**Value**
Random seed	0
Number of training steps	162000
Batch size	64
Checkpoint save frequency (steps)	2000
Predictor learning rate	5×10^{-5}
Action encoder learning rate	2×10^{-4}
Latent action model learning rate	2.5×10^{-5}
Optimizer	AdamW
β_1	0.9
β_2	0.999
ϵ	10^{-8}
Weight decay	0.01
AMSGrad	False
Normalize action inputs	True
Action scaling factor	100.0
Frame skip	5
Input image resolution	224×224
Predictor enabled	True
Decoder enabled	False
Planner action representation	Raw
Training mode flag	True
Train encoder	False
Train predictor	True
Train decoder	False
Action repetition factor	1
Number of historical context	1
Prediction horizon	1
Feature concatenation dimension	1

Model variants are instantiated through a small set of gating parameters that enable or disable specific architectural components. These gates determine the action representation provided to the predictor during training and the action representation used during planning.

The supported action-representation regimes include: (i) explicit action embeddings obtained from ground-truth action labels, (ii) continuous latent actions inferred from consecutive observations, and (iii) discrete latent actions obtained via vector quantization.

Certain gate combinations are mutually exclusive. Each variant inherits the base architecture,

optimization procedure, and training configuration. For transparency, training-time and planning-time configurations are reported separately whenever the active components differ between these phases. Figure 4 gives an overview of the varaints.

Table 4: Overview of model variants and their active components. IDM denotes the inverse dynamics model. Used action types: [1] embedded ground truth action, [2] continuous latent action, [3] quantized action vector from the codebook.

Variant	Used action	Action encoder	IDM	VQ	Planner variable
Baseline	[1]	On	Off	Off	Raw
LA	[2]	Off	On	Off	Latent
LA-VQ	[3]	Off	On	On	Discrete

Latent Action VQ Variant (LA-VQ). The latent-action model with vector quantization replaces explicit action conditioning with a discrete latent action representation obtained via vector quantization over inferred latent actions. The training of the latent-action VQ model required approximately 89 hours.

Table 5 reports the training-time configuration, while Table 6 summarizes the planning-time parameters.

Table 5: Variant specific training parameters of the Latent Action VQ Model

Parameter	Value
Use action encoder	False
Use latent-action model	True
Use vector quantization	True
Train action encoder	False
Train latent-action model	True
Codebook splits	1
Codebook embedding dimension	32
Latent action dimensionality	32
Number of codebook entries	32
VQ commitment cost	0.05
Exponential moving average decay	0.99
Two-phase training enabled	False
Two-phase warmup steps	0
Two-phase mixing steps	0

Table 6: Variant specific planning parameter of the Latent Action VQ Model

Parameter	Value
Planner	Discrete CEM
Use action encoder	False
Use latent-action model	True
Use vector quantization	True
Planned action representation	Discrete
Pseudocount	1.0
Momentum	0.7
Minimum probability	0.001
Initialization smoothing	0.05
Sampling temperature	1.5
Use latent action	True
Use latent state	True
Use state delta	False
Return Mode	False

In addition to the encoder and predictor, this variant introduces an inverse dynamics model based on the spatio-temporal transformer architecture proposed by Gao et al. (2025), together with a vector-quantization codebook. Implementation details of the inverse dynamics model are reported in Table 7. We implement an EMA-based codebook that supports splitting the input into multiple partitions, each of which is quantized independently. Due to simplicity we use a single standard codebook and do not use the splitting functionality. Codebook configuration details are provided in Table 5.

Table 7: Inverse Dynamics Model Implementation Details

Parameter	Value
Encoder blocks	12
Decoder blocks	0
Number of attention heads	6
Dropout	0.0

We illustrate the active components of this variant and the corresponding configuration parameters in Figure 4 for training and in Figure 5 for planning.

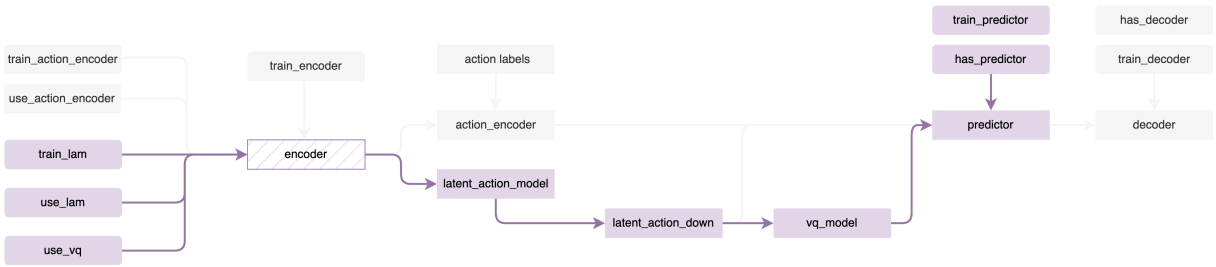

Figure 4: Active model components and configuration parameters for the Latent-Action VQ variant during training. Rounded boxes on the left denote configuration flags. Rectangular boxes represent model components. Hatched elements indicate frozen components. Violet highlights denote active components and flags, while light grey indicates disabled or unused components and flags.

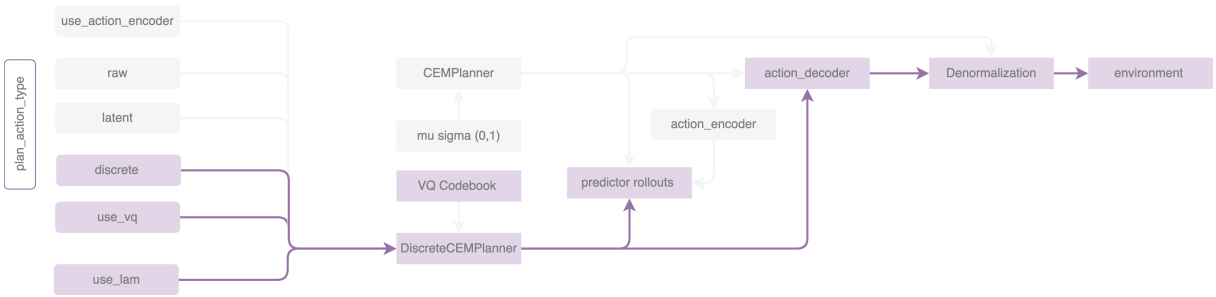

Figure 5: Active model components and configuration parameters for the Latent-Action VQ variant during planning. Rounded boxes on the left denote configuration flags. Rectangular boxes represent model components. Hatched elements indicate frozen components. Violet highlights denote active components and flags, while light grey indicates disabled or unused components and flags.

Latent Action Variant (LA). The latent-action variant employs a continuous latent action representation inferred from consecutive observations, without vector quantization. Variant-specific training parameters are reported in Table 8, while planning parameters are summarized in Table 9. Training of the latent-action model required approximately 86 hours.

Table 8: Variant specific training parameters of the Latent Action Model.

Parameter	Value
Use action encoder	False
Use latent-action model	True
Use vector quantization	False
Train action encoder	False
Train latent-action model	True
Latent action dimensionality	32

Table 9: Variant specific planning parameters of the Latent Action Model.

Parameter	Value
Subplanner	CEM
Use action encoder	False
Use latent-action model	True
Use vector quantization	False
Planned action representation	Latent
Use prior	False
Use latent action	True
Use latent state	True
Use state delta	False

The latent-action variant is identical to the latent-action VQ variant, except that vector quantization is omitted. Details of the inverse dynamics implementation are provided in Table 7. We illustrate the active components of this variant and the corresponding configuration parameters in Figure 6 for training and in Figure 7 for planning.

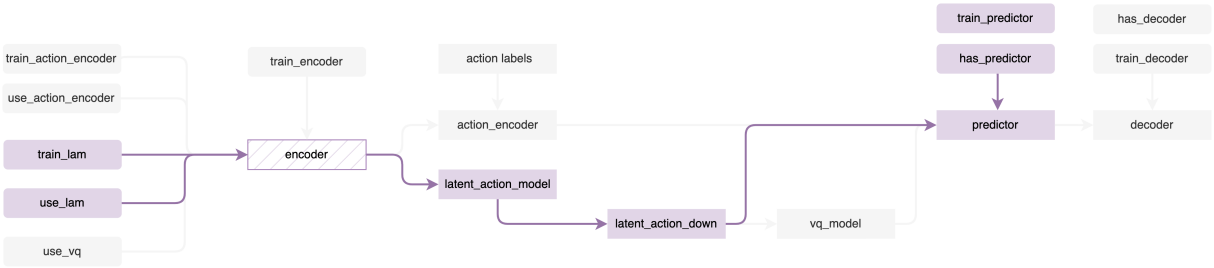

Figure 6: Active model components and configuration parameters for the latent-action variant during training.Rounded boxes on the left denote configuration flags. Rectangular boxes represent model components. Hatched elements indicate frozen components. Violet highlights denote active components and flags, while light grey indicates disabled or unused components and flags.

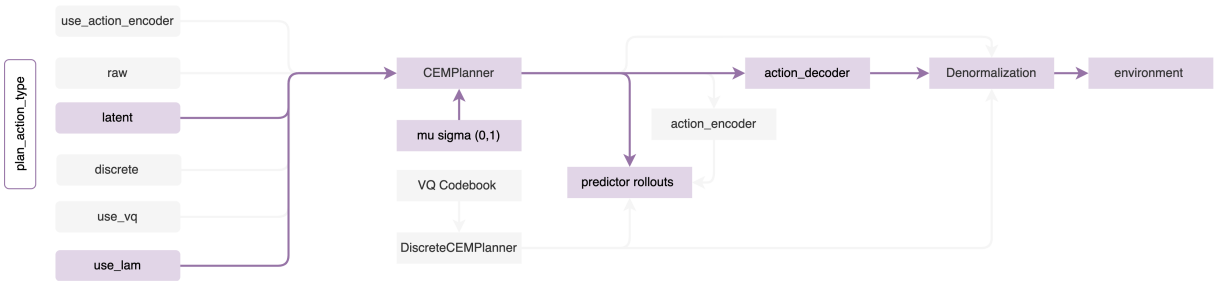

Figure 7: Active model components and configuration parameters for the latent-action variant during planning. Rounded boxes on the left denote configuration flags. Rectangular boxes represent model components. Hatched elements indicate frozen components. Violet highlights denote active components and flags, while light grey indicates disabled or unused components and flags.

Baseline. The baseline model conditions the predictor on explicit action embeddings derived from ground-truth action labels via an action encoder implemented as a one-dimensional convolution with kernel size one and stride one. No latent-action inference or vector quantization is

used. The training-time and planning-time configurations for this variant are summarized in Tables 10 and 11. Training of the baseline model required approximately 32 hours.

Table 10: Baseline configuration parameters (training-time).

Parameter	Value
Use action encoder	True
Use latent-action model	False
Use vector quantization	False
Train action encoder	True
Train latent-action model	False
Action embedding dimension	32

Table 11: Baseline planning configuration parameters (planning-time).

Parameter	Value
Planner	CEM
Use action encoder	True
Use latent-action model	False
Use vector quantization	False
Planned action representation	Raw
Use prior	False

We illustrate the active components of this variant and the corresponding configuration parameters in Figure 8 for training and in Figure 9 for planning.

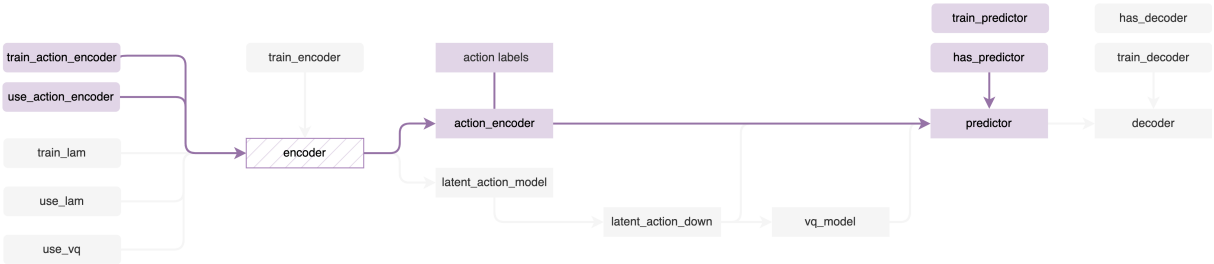

Figure 8: Active model components and configuration parameters for the baseline during training. Rounded boxes on the left denote configuration flags. Rectangular boxes represent model components. Hatched elements indicate frozen components. Violet highlights denote active components and flags, while light grey indicates disabled or unused components and flags.

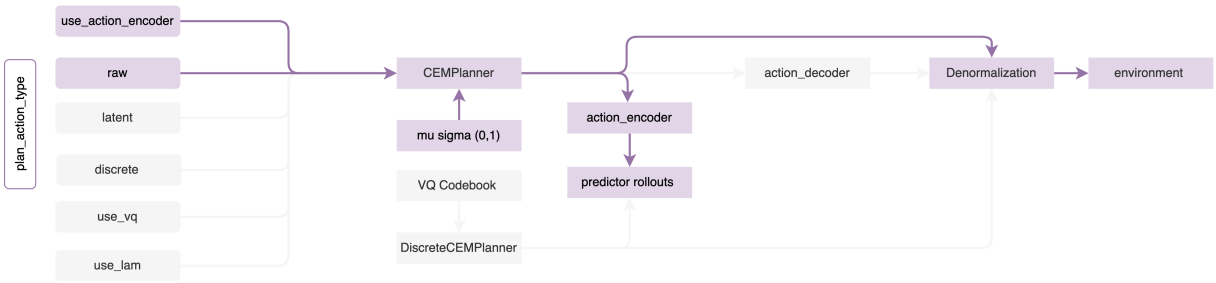

Figure 9: Active model components and configuration parameters for the Baseline during planning. Rounded boxes on the left denote configuration flags. Rectangular boxes represent model components. Hatched elements indicate frozen components. Violet highlights denote active components and flags, while light grey indicates disabled or unused components and flags.

Planning Implementation. Planning is performed using the Cross-Entropy Method (CEM) implementation form (Zhou et al., 2024) and a discrete variant of our own, depending on the action space. The planner samples candidate action sequences, evaluates them under the learned world model, and executes the full action sequence of the selected plan. While variant-specific planning configurations are reported for each model Table 12 summarizes the planning parameters shared across all experiments.

Table 12: Fixed planning and evaluation configuration used in all planning experiments.

Parameter	Value
Random seed	0
Number of evaluation episodes	100
Goal source	Dataset
Goal horizon (H)	5
Number of plotted samples	0
Training mode	Disabled
Planning weight (α)	1
Objective function base	2
Objective function mode	Last
Planning horizon	5
Top-k samples	30
Number of sampled trajectories	300
Sampling variance scale	1
Optimizer steps per iteration	10
Evaluation frequency (iterations)	1
Use mode as first sample	No

The Action Decoder. We implement the action decoder as a lightweight `torch.nn.Module` that receives the patch token sequences of the latent state z_t and compresses it into a single vector using learned-query attention pooling. The resulting representation is layer-normalized and projected into the 128-dimensional decoder space via a small MLP composed of a Linear layer,

a GELU activation, and a second Linear layer. The projected features are concatenated with the latent action and processed by a final MLP head, consisting of LayerNorm followed by three Linear layers with GELU activations, to predict the ground-truth action vector. Throughout all experiments, Δe is not used as an input, and the optional output bounding mechanism is not applied. Table 13 provides the decoder training and evaluation configurations.

Table 13: Training and evaluation configuration for the Action Decoder.

Parameter	**Value**
Optimizer	AdamW
Learning rate	1×10^{-3}
Weight decay	0.0
Loss type	Huber
Huber δ	1.0
Batch size	256
Training pairs	100,000
Validation pairs	10,000
Max epochs	200
Early stopping patience	5
Early stopping min. Δ	1×10^{-4}
Use state feature (e_t)	True
Use delta feature (Δe)	False
Use latent action	True

5.5 Evaluation Protocol

All evaluation metrics are computed on the validation split specified in Section 5.3. Metrics are recomputed from the final trained checkpoints using a fixed evaluation script to ensure consistent comparison across model variants. The evaluation code is publicly available at https://github.com/pythonpedro99/dino_wm_latent_actions. Unless stated otherwise, reported values correspond to the mean and standard deviation over the evaluation samples specified by the configuration. All numerical configuration parameters used for training, planning, and evaluation are defined in the Implementation Section under 5.4. Evaluation is deterministic given a fixed random seed, and no model parameters are updated during evaluation.

5.6 Experiments

This section describes the experimental procedure used to generate the results reported in Section 6. We evaluate three model variants, Baseline, LA and LA-VQ. Model architectures and all fixed configuration parameters are specified in Section 5.4. Unless stated otherwise, all results are computed according to the evaluation protocol defined in Section 5.5.

Experiment 1: Prediction Quality We evaluate predictive accuracy for all final model variants and report next-step prediction error and open-loop rollout error. The resulting prediction

metrics are summarized in Table 14.

Experiment 2: Action Decoding Quality Planning with latent action models requires mapping them to environment actions. We therefore train and evaluate action decoders for LA and LA-VQ. Specifically, we compute the root mean squared error (RMSE) using 5, 10, 25, 50, and 100 thousand samples to assess how decoder performance scales with training set size, and we report the mean-action baseline for comparison. We further conduct ablations in which the decoder predicts actions from the latent action alone, the latent action–state pair, and the state alone. The decoder architecture, optimization settings, and the decoder configuration used during planning are fixed as specified in Section 5.4. While for the ablation we only change Training pairs, Use state feature and Use latent action. Validation accuracy as a function of decoder training set size is reported in Table 15 and 16. Hyperparameters are held constant across dataset sizes. Decoder training and evaluation are conducted on a single NVIDIA L4 GPU and completed in under 3 hours.

Experiment 3: Planning Performance We evaluate end-to-end planning performance of the final models under the fixed planning setup defined in Section 5.4. The baseline plans directly in the ground-truth action space, whereas LA and LA-VQ plan in latent-action space and execute actions via the corresponding trained decoder. We report success rate, mean state distance, and pixel ℓ_2 error in Table 17. Planning is conducted on a single NVIDIA A100 GPU and completed in less then 2 hours.

Experiment 4: Latent Action Collapse Diagnostics To verify that latent actions remain non-degenerate, we apply two diagnostic tests defined in Section 5.2. The information-bypass (z-swap) test and the action-ignoring (action-shuffle) test. These measures allow us to detect whether the latent action is exploited for shortcut learning or ignored by the forward dynamics model. The diagnostics are reported in Table 18. For LA-VQ, we additionally report codebook utilization statistics as defined in Section 5.2.

6 Results

In the following, we report the results of our experiments. We refer to the action-supervised model as the Baseline. LA denotes our latent-action variant, which replaces ground-truth actions with a continuous latent action inferred by an inverse dynamics model and constrained only via dimensionality reduction. LA-VQ denotes the variant in which the latent action is additionally vector-quantized, yielding a discrete latent action representation.

6.1 Prediction Quality

Table 14: Prediction Quality on the Validation Set. Next-Step MSE evaluates single-step latent prediction accuracy. Rollout MSE measures open-loop latent prediction error over 600 seeded random-length rollouts. Values are reported as mean ± standard deviation. We highlight the best results.

Model	Next-Step MSE ↓	Rollout MSE ↓
Baseline	0.064 ± 0.006	0.28 ± 0.26
LA	**0.030** ± 0.002	**0.23** ± 0.14
LA-VQ	0.068 ± 0.005	0.95 ± 0.48

As shown in Table 14, among the evaluated models, the LA variant achieves the lowest Next-Step MSE, which is 53% lower than the Baseline, and the lowest Rollout MSE, which is 18% lower than the Baseline. The highest Next-Step MSE is observed for the LA-VQ variant, which is 6% higher than the Baseline, while its Rollout MSE exceeds the Baseline by 240%. Regarding variability, the standard deviations for Next-Step MSE are similarly small across models, whereas Rollout MSE standard deviations are much larger and vary more across the variants.

6.2 Action Decoding Quality

Table 15: Action decoding performance (RMSE) of the LA variant on the validation split for different decoder input configurations and training set sizes. Lower is better. A mean-action baseline achieves an RMSE of 1.44.

Decoder Inputs	5k	10k	25k	50k	100k
$\tilde{u}$	1.07	0.98	0.90	0.83	0.79
z	1.20	1.13	1.04	1.00	0.95
$\tilde{u}, z$	**0.77**	**0.70**	**0.62**	**0.57**	**0.50**

Table 15 reports action-decoding performance for three decoder input configurations $\tilde{u}$, z, and the combined input $(\tilde{u}, z)$ as the number of training pairs increases. Here, $\tilde{u}$ denotes the latent action, and z the latent observation representation. All configurations outperform the mean-action baseline (RMSE = 1.44) at every training-set size, and RMSE decreases monotonically as more training data are used. Across all dataset sizes, $(\tilde{u}, z)$ consistently achieves the lowest RMSE and yields the best overall result, with a 65.2% reduction relative to the baseline, while $\tilde{u}$ alone outperforms z alone throughout.

Table 16: Action decoding performance (RMSE) of the LA-VQ varaint on the validation set for different decoder input configurations and training set sizes. Lower is better. A mean-action baseline achieves an RMSE of 1.44.

Decoder Inputs	5k	10k	25k	50k	100k
u	1.39	1.39	1.39	1.39	1.39
z	1.24	1.15	1.02	1.00	0.95
u, z	**0.89**	**0.85**	**0.76**	**0.72**	**0.67**

Table 16 reports action-decoding performance for three decoder input configurations—u, z, and the combined input (u, z)—as the number of training pairs increases. Here, u denotes the quantized latent action, and z the latent observation representation. All configurations outperform the mean-action baseline (RMSE = 1.44) at every training-set size. RMSE decreases monotonically with more training data for z and (u, z), whereas u alone shows almost no scaling, remaining at 1.39 throughout. Across all dataset sizes, (u, z) consistently achieves the lowest RMSE and yields the best overall result, with a 53.5% reduction relative to the baseline. Moreover, combining u with z provides a substantial improvement over z alone at every training-set size.

Across all training-set sizes, both variants achieve their best action-decoding performance when the decoder receives both the action and observation latents. Comparing the two variants directly, LA attains a lower RMSE than LA-VQ in this combined-input setting at every dataset size. When decoding from the action latent alone, performance improves with more training data for LA, whereas it remains constant for LA-VQ across all sizes. As expected, the z-only results are nearly identical between variants at each training-set size. Unless stated otherwise, all subsequent experiments use the combined-input configuration trained on the largest training set.

6.3 Planning Performance

Table 17: Planning performance of all variants for the last iteration, reported as the mean over 100 different evaluation tasks drawn from the validation split. SR denotes the fraction of successful trajectories. StateDist is the mean ℓ_2 distance between the final and goal states in ground-truth state space. PixL2 is the mean ℓ_2 distance between final and goal observations in pixel space.

Model	SR ↑	StateDist ↓	PixL2 ↓
Baseline	**0.52**	**47.82**	**3.00**
LA-VQ	0.19	51.90	4.13
LA	0.02	110.02	4.89

As reported in Table 17 the Baseline achieves the highest SR with 52 %, followed by LA-VQ with 19 %, while LA attains a near zero SR. A similar ordering is observed for StateDist, where the Baseline yields the lowest distance, LA-VQ is higher but close, and LA shows the largest distance. For PixL2, the Baseline again obtains the lowest value, followed by LA-VQ and LA.

6.4 Latent Action Diagnostics

Table 18: Latent action diagnostics of the final models on the validation set. **z-swap** reports the fraction of swaps where the resampled latent action code effectively selects another sample's next state, i.e., $f(z_t^{(i)}, u_t^{(j)})$ is closer to $z_{t+1}^{(j)}$ than to $z_{t+1}^{(i)}$. $\Delta_{\textbf{shuf}}$ reports the factor by which the prediction error increases when actions are shuffled across samples. PPL_{norm} is the normalized codebook perplexity, while **Dead rate** is the fraction of codebook entries that are never selected.

Model Variant	z-swap score	Δ_{shuf}	$\text{PPL}_{\text{norm}} \uparrow$	Dead rate $\downarrow$
Baseline	0.01	4.08	–	–
LA	0.18	14.76	–	–
LA-VQ	0.00	2.51	0.90	0.0

We treat the baseline as a healthy reference for the measures reported in Table 18, since it is trained on ground-truth action labels. We refer to Section 5.2 for a more detailed explanation of the swap metrics. The baseline's z-swap score is near zero, indicating that swapping latent actions almost never causes the predictor to align more closely with another sample's next-state latent, while shuffling actions increases the prediction error by roughly 4×.

The LA variant deviates strongly from this reference. Its z-swap score increases to 0.18, meaning that in 18% of swaps the resampled latent action code selects another sample's next state. In addition, LA exhibits substantially higher sensitivity to action shuffling, with a nearly 15× increase in error relative to the unshuffled condition. This is markedly larger than for the other variants.

For LA-VQ, the z-swap score is effectively zero, matching the near-zero behavior of the baseline in this diagnostic. Action shuffling degrades performance by roughly 2.5×, which is relatively close to the baseline. Regarding codebook utilization, $\text{PPL}_{\text{norm}} = 0.90$ indicates high normalized perplexity, and all codebook entries are selected at least once on the validation set.

7 Discussion

Overview. In this chapter, we discuss the results presented in Section 6 in the context of our research question. Specifically, we investigated whether the latent-action-conditioned variants of DINO-WM can match the planning performance of the original action-supervised baseline on PushT when trained solely on image-only trajectories. Empirically, this does not hold. Our proposed variants perform below the baseline, which achieves SR = 0.52, while LA-VQ reaches SR = 0.19 and the LA variant SR = 0.02 (Table 17). At the same time, predictive metrics are not aligned with planning outcomes. While baseline attains a Next-Step MSE = 0.064 the LA variant achieves even lower Rollout MSE = 0.030 under our evaluation protocol (Table 14), yet fails at planning. LA exhibits a high z-swap rate (0.18), whereas the LA-VQ variant restores baseline-like z-swap behavior (≈ 0) (Table 18).

In the following, we interpret our results in detail, provide the most likely mechanisms behind them, and outline remaining investigations and potential mitigation strategies. We relate our

findings to prior work and summarize the implications. Finally, we state the limitations and propose future work.

7.1 Baseline as a reference

We use the baseline as an empirical reference point for planning performance and as a sanity check for latent-action diagnostics, against which all latent-action variants are interpreted. This allows us to attribute differences in downstream performance to the structural changes introduced by latent actions, rather than to deficiencies in the overall training or evaluation setup. The baseline achieves a success rate of 0.52, which we adopt as the comparison anchor for our proposed variants. Furthermore, action resampling never selects another sample's next state, providing a sanity check against shortcut learning. Action shuffling induces an approximately $4\times$ increase in prediction error, indicating strong action dependence and serving as a reference for the latent-action variants (Table 18).

Finally, as reported in Table 14, the baseline exhibits a relatively high rollout error, with a mean MSE of 0.28 and a variance of 0.26. This variance is largely attributable to the evaluation protocol, which uses open-loop rollouts of random length, naturally amplifying error accumulation and variability. Consequently, we avoid overinterpreting the absolute rollout variance in the subsequent discussion and instead focus on relative changes across model variants.

7.2 Low prediction error via a shortcut does not yield controllable dynamics

The central failure mode of the LA variant is that a weak bottleneck on the inferred latent action allows the inverse dynamics model (IDM) to transmit information about the prediction target. Because the IDM observes (z_t, z_{t+1}) during training, it can encode features of z_{t+1} into $\tilde{u}_t$, thereby reducing $\mathcal{L}_{\text{pred}}$ without requiring the forward dynamics model (FDM) to learn a compositional, action-conditioned transition operator. Our diagnostics are strongly consistent with this interpretation. The IDM variant without vector quantization achieves the best prediction quality in Table 14, substantially improving both next-step and rollout MSE relative to the baseline, while its planning success collapses to near zero in Table 17. The latent-action diagnostics in Table 18 further support the shortcut explanation: compared to the baseline, the model exhibits an approximately $18\times$ higher z-swap score and a markedly larger degradation under action shuffling than the other variants.

Next-step and rollout prediction metrics evaluate the model under the IDM-induced latent-action distribution, whereas planning optimizes over counterfactual latent-action sequences that may be rare or unsupported under training. If the learned dynamics are only coherent on-support, then CEM can exploit spurious latent-action codes that reduce the latent goal cost in-model but do not correspond to realizable or compositional controls in the environment after decoding. The same distribution-shift argument also applies to the action decoder, which is trained on dataset latents but queried during planning on imagined states produced by model rollouts. Importantly, the fact that the latent action remains statistically predictive of the ground-truth action (Table 15) does not contradict this account.

Decodability from $(z_t, \tilde{u}_t)$ is a weak requirement. It does not imply that optimized latent-action sequences correspond to feasible or temporally coherent control, nor that they remain valid under closed-loop execution. Moreover, these issues may be amplified by an unstable forward dynamics model resulting from shortcut learning.

Overall, these results indicate that low next-state MSE under the IDM's latent-action distribution does not guarantee planning success. Consistent with prior work on latent-action world models (Schmidt et al., 2023; Ye et al., 2024b), our findings support the need for an explicit information bottleneck.

7.3 Vector quantization prevents leakage but weakens long-horizon fidelity

Vector quantization largely eliminates the shortcut signature in the action channel. The LA-VQ variant attains z-swap ≈ 0, consistent with baseline behavior, while maintaining high codebook usage ($\text{PPL}_{\text{norm}} = 0.90$ and no dead codes), suggesting that the discrete bottleneck prevents action tokens from carrying next-state-identifying information and avoids code collapse (Table 18).

However, this stabilization coincides with substantially worse long-horizon predictive fidelity under our rollout metric (Rollout MSE = 0.95 vs. 0.28 for the baseline), despite only a small degradation in one-step error (Next-Step MSE = 0.068 vs. 0.064; Table 14). A plausible explanation is that the learned discrete codes do not act as globally consistent control primitives. Instead, their semantics are state-dependent. This is consistent with the decoding results. Decoding from u alone is near a mean-action baseline and does not improve with more labeled transitions, while conditioning the decoder on both (u, z) yields large gains and scales with data (Table 16). Under state-dependent semantics, repeated application of the same code across imagined states can become inconsistent, which would disproportionately harm open-loop rollouts and planning horizons relative to one-step teacher-forced prediction.

Importantly, the shuffling diagnostic also suggests that LA-VQ may be less action-dependent than the baseline: $\Delta_{\text{shuf}} = 2.51$ vs. 4.08 (Table 18). One interpretation is that discretization regularizes the model toward state-dominant prediction, which can make the latent action channel more stable yet reduce its effective causal influence on transitions.

Additional factors may contribute to the long-horizon degradation. First, latent actions are learned rather than given. Under a one-step prediction objective, the discrete representation may be optimized for immediate next-state prediction without enforcing long-horizon consistency of the induced control primitives. Second, vector quantization introduces discretization error relative to fine-grained continuous action labels, and such error can accumulate over time under open-loop rollout. Finally, the observed predictive cost may partially reflect suboptimal quantizer settings, rather than an inherent drawback of discretization. Disentangling these effects would require targeted ablations beyond the scope of this work.

Across the two variants, performance appears sensitive to the strength of the action information bottleneck. Too weak permits target leakage and destroys controllability, while too strong stabilizes training but may reduce long-horizon causal influence and control fidelity. This suggests that learning controllable latent actions requires not merely preventing leakage, but balancing

information sufficiency with semantic consistency across states.

7.4 Why planning remains below the baseline even when latent actions are stabilized

Although LA-VQ reaches non-trivial performance (SR = 0.19), it remains far below the baseline (SR = 0.52; Table 17). The fact that StateDist is relatively close to the baseline (51.90 vs. 47.82) but PixL2 is noticeably worse (4.13 vs. 3.00) suggests that the model often makes partial progress toward the goal but fails to reliably complete the task, potentially due to compounding errors.

A second likely bottleneck is the latent-action decoder. The decoder is trained on state–code pairs induced by the dataset distribution, while execution under planning can visit states that are out-of-distribution with respect to the decoder's training set. Even with 100k labeled transitions, the best RMSE for LA-VQ with (u, z) inputs is 0.67 (Table 16), leaving substantial room for error accumulation over time. This is not a concern for the baseline, which does not require a decoder. Therefore, the planning gap for LA-VQ conflates at least two factors. The quality and controllability of the learned latent-action-conditioned world model, and the quality of the learned low-level decoding policy that maps codes to environment actions.

Finally, our planning implementation for latent actions uses discrete CEM with a fixed compute budget across methods. While this ensures fairness, it may disadvantage the latent-action setting if the effective search space is harder and future work should scale the search budget to test whether compute is a limiting factor.

7.5 Discrete latent actions behave like policy inputs, not standalone action labels

Our experiments indicate that the vector-quantized latent actions learned in our setup are not state-agnostic action labels that can be decoded in isolation. Instead, the discrete codes become useful only when provided jointly with the current latent state, enabling decoding back to continuous controls and consistently outperforming a state-only decoder.

Table 16 shows that u alone performs near a mean-action baseline and does not scale with more data, whereas (u, z) substantially improves over z alone for all dataset sizes. This indicates that discrete codes are best interpreted as state-conditioned modes that disambiguate behavior given the current state representation, rather than as state-agnostic action labels.

In deployment, such codes should therefore be coupled with a learned low-level policy and evaluated as part of a hierarchical control stack, not as an interpretable action set on their own.

7.6 Limitations

While our empirical results provide evidence for controllable behavior under latent action conditioning, several methodological, computational, and interpretational constraints limit the strength and scope of our conclusions. We discuss these limitations below.

We use rollout MSE primarily as a comparative proxy for world-model quality under test-time

optimization. However, the evaluation protocol has a structural mismatch. Latent actions are inferred from the true trajectory, whereas only the first predicted step is conditioned on the corresponding true latent state. Subsequent steps apply the same codes to imagined latent states. If code semantics are state-dependent, this could change the meaning of the latent action sequence, degrading long-horizon rollouts while one-step error remains low.

Beyond this methodological limitation, our results are also shaped by practical computational constraints. We trained all variants for roughly 10× fewer steps than the original work (Zhou et al., 2024), which likely limits absolute long-horizon prediction quality. Especially for LA variants, the latent action is learned jointly during training, which can be noisy early on and may require longer training to stabilize compared to the ground-truth action conditioning setup. All methods were also evaluated under a restricted planning budget. We enforced identical planning compute across variants to enable controlled comparison. However, LA-VQ relies on a discrete CEM formulation whose effective sampling budget may not be directly comparable to the continuous CEM setting.

A further limitation concerns generalization and the role of action supervision. Although the architecture is intended to be task- and environment-agnostic and leverages a pretrained DINOv2 encoder, we trained and evaluated in a single environment. We therefore cannot claim robustness across varying observation statistics, camera geometries, object distributions, or dynamics regimes. Latent-code controllability may partially rely on environment-specific regularities that do not transfer beyond the current setting. While the world model is trained on image-only trajectories, deployment still requires a learned action decoder trained on labeled action pairs. In our experiments, decoder error remained non-negligible at 100k labeled samples, suggesting that reliable decoding may require substantially more supervision.

Finally, our analysis remains limited in terms of mechanistic understanding. We rely on diagnostics that detect coarse failure signatures, e.g., latent-action ignoring or target leakage, but we do not characterize the structure or stability of individual latent codes. We therefore cannot determine whether a code has consistent semantics across states, whether repeated codes yield stable and compositional effects, or whether behavior drifts as imagined states move off-support. Without targeted intervention-based probes, these mechanisms remain entangled and prevent precise causal attribution of controllability failures.

7.7 Future Work

Our results suggest that controllability is entangled with several possible mechanisms, ranging from insufficient planning configuration and non-negligible decoding error to remaining VQ ablations, limited mechanistic understanding of the latent action space, and potentially misleading training objectives. Potential future work therefore splits naturally into two directions. First, thesis-bound follow-ups will directly address these entanglements under controlled compute constraints. Second, broader extensions will expand the scope beyond this thesis.

Since we did not exhaustively scale the decoder, a logical next step would be to expand decoder training in order to determine whether downstream control is primarily bottlenecked by decoder

error and to establish the achievable performance ceiling for how accurately latent actions can be decoded into action labels. While decoder scaling addresses one potential bottleneck, controllability may also be limited by the planning setup itself. To disentangle these factors, a controlled sweep over planning budget and key hyperparameters such as CEM population size, number of iterations, elite fraction, noise/temperature schedules are required. To mitigate the effects of fixed-compute comparison, a compute–performance analysis could reveal whether different variants scale differently with planning compute, and avoid conflating method quality with an arbitrary compute allocation.

Furthermore, it remains an open question how to precisely characterize the structure and semantics of the latent action space—specifically, what the codes encode, how their effects differ across states, and whether they exhibit compositional structure. Targeted probes, such as applying fixed codes across state clusters, testing repeatability, and measuring effect drift as the rollout horizon increases, as well as simple intervention-style tests and visualizations, could provide mechanistic insight and inform subsequent ablations. Building on these insights, ablations over VQ settings are inevitable to understand expressivity and stability trade-off, for example variing the codebook size and compare structured discrete spaces, such as factorized codebooks. The goal is to determine whether richer discrete actions improve planning without inducing semantic collapse, state dependence, or instability under off-support rollouts.

Finally, addressing the observed decoupling between one-step error and long-horizon rollout quality, future work should systematically compare training and evaluation setups that emphasize one-step prediction, multi-step prediction, or a combination of both. This would directly test whether improved rollout consistency requires explicitly optimizing multi-step behavior, and whether a combined objective better predicts control-relevant performance.

Going beyond the scope of this thesis, a logical next step would be a scaling study over training steps and approaching the original regime. This would clarify whether long-horizon errors are primarily data- or compute-limited, and whether performance trends are smooth or saturating. This would also allow comparison against alternative baselines such as DreamerV3, TD-MPC-2 or IRIS (Hafner et al., 2023; Hansen et al., 2023; Micheli et al., 2022). Crucially, such a scaling study should be paired with evaluation on at least a small environment suite e.g., the DINO-WM environments, to test robustness across observation statistics and dynamics regimes. This directly targets whether latent-action semantics and planning behavior transfer beyond the single environment studied here.

As a complementary direction, replacing the DINOv2 encoder with a predictive video encoder such as V-JEPA-2 (Assran et al., 2025) constitutes a particularly promising avenue. In contrast to contrastive or reconstruction-based image encoders, V-JEPA-2 is trained directly on video data under a predictive objective, potentially yielding latent representations whose geometry is more naturally aligned with world-model learning and long-horizon planning.

8 Conclusion

In this thesis, we investigated whether a latent-action variant of DINO-WM, trained jointly with an inverse dynamics model (IDM) and a forward dynamics model directly in DINO embedding space, can match the planning performance of the original action-supervised baseline on PushT. Under our training regime, it does not. The baseline achieves a success rate of 52%, whereas our continuous latent-action variant reaches 2% and our vector-quantized latent-action variant reaches 19%.

Our experiments indicate that the main obstacle lies in controllability and deployment, not merely one-step predictive accuracy. The unconstrained continuous latent-action model exhibits clear shortcut behavior. During training, the IDM can leak target information into the latent action, yielding low prediction error while producing dynamics that fail under counterfactual action optimization at planning time. Introducing vector quantization effectively mitigates this leakage and restores baseline-like shortcut diagnostics. However, planning performance remains substantially below the baseline, consistent with degraded long-horizon fidelity and state-dependent code semantics.

Beyond the headline performance gap, we obtain several mechanistic insights. First, latent actions learned under a one-step prediction objective with joint IDM/FDM training are not state-agnostic. They become meaningfully decodable into ground-truth actions primarily when conditioned on the current state. Second, our results suggest a mismatch between one-step training objectives and the requirements of model-based planning with optimized action sequences. Predictive metrics computed under the IDM-induced latent-action distribution can be poorly aligned with downstream planning success.

These findings are tempered by practical constraints. Time and compute limitations prevented systematic ablations of codebook design and planning hyperparameters, and limited training scale may disproportionately affect latent-action models that must learn both the action representation and the dynamics jointly. In addition, deployment in the latent action setting requires an action decoder. Resulting decoding error, as well as distribution shift between dataset states and imagined rollouts, likely contribute to the remaining performance gap.

Overall, our results support prior evidence that explicit information restriction is necessary in IDM/FDM latent-action setups, and they highlight that preventing leakage alone is insufficient. Learned codes must also induce stable and consistent control semantics over long horizons and under planning-time optimization. This motivates future work on systematic vector-quantization and bottleneck ablations, training objectives that more directly target multi-step consistency, and scaling studies and compute-performance analyses that disentangle model capacity, planning budget, and decoder limitations. If these challenges can be addressed, this line of work could yield world models that are pretrained on unlabeled, internet-scale video data and learn an embodiment-agnostic latent action space, enabling rapid deployment through environment-specific decoding.

References

Assran, Mahmoud et al. (Jan. 2023). *Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture*. DOI: 10.48550/arXiv.2301.08243. arXiv: 2301.08243 [cs]. (Visited on 02/08/2026).

Assran, Mido et al. (June 2025). *V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning*. DOI: 10.48550/arXiv.2506.09985. arXiv: 2506.09985 [cs]. (Visited on 06/16/2025).

Bardes, Adrien, Jean Ponce, and Yann LeCun (May 2021). *VICReg: Variance-Invariance-Covariance Regularization for Self-Supervised Learning*. DOI: 10.48550/arXiv.2105.04906. arXiv: 2105.04906 [cs]. (Visited on 03/21/2025).

Bengio, Yoshua, Aaron Courville, and Pascal Vincent (Aug. 2013a). "Representation Learning: A Review and New Perspectives". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 35.8, pp. 1798–1828. ISSN: 1939-3539. DOI: 10.1109/TPAMI.2013.50. (Visited on 02/14/2026).

Bengio, Yoshua, Nicholas Léonard, and Aaron Courville (Aug. 2013b). *Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation*. DOI: 10.48550/arXiv.1308.3432. arXiv: 1308.3432 [cs]. (Visited on 02/14/2026).

Bruce, Jake et al. (May 2025). *Genie: Generative Interactive Environments*. DOI: 10.48550/arXiv.2402.15391. arXiv: 2402.15391 [cs]. (Visited on 10/28/2025).

Bu, Qingwen et al. (May 2025). *UniVLA: Learning to Act Anywhere with Task-centric Latent Actions*. DOI: 10.48550/arXiv.2505.06111. arXiv: 2505.06111 [cs]. (Visited on 10/28/2025).

Caron, Mathilde et al. (June 2020). *Unsupervised Learning of Visual Features by Contrasting Cluster Assignments*. DOI: 10.48550/arXiv.2006.09882. arXiv: 2006.09882 [cs]. (Visited on 12/27/2025).

Caron, Mathilde et al. (Apr. 2021). *Emerging Properties in Self-Supervised Vision Transformers*. DOI: 10.48550/arXiv.2104.14294. arXiv: 2104.14294 [cs]. (Visited on 12/27/2025).

Chen, Chang et al. (Feb. 2022). *TransDreamer: Reinforcement Learning with Transformer World Models*. DOI: 10.48550/arXiv.2202.09481. arXiv: 2202.09481 [cs]. (Visited on 04/15/2025).

Chen, Ting et al. (Feb. 2020a). *A Simple Framework for Contrastive Learning of Visual Representations*. DOI: 10.48550/arXiv.2002.05709. arXiv: 2002.05709 [cs]. (Visited on 12/27/2025).

Chen, Xinlei and Kaiming He (Nov. 2020b). *Exploring Simple Siamese Representation Learning*. DOI: 10.48550/arXiv.2011.10566. arXiv: 2011.10566 [cs]. (Visited on 01/02/2026).

Chi, Cheng et al. (Mar. 2023). *Diffusion Policy: Visuomotor Policy Learning via Action Diffusion*. DOI: 10.48550/arXiv.2303.04137. arXiv: 2303.04137 [cs]. (Visited on 12/12/2025).

De Boer, Pieter-Tjerk et al. (Feb. 2005). "A Tutorial on the Cross-Entropy Method". In: *Annals of Operations Research* 134.1, pp. 19–67. ISSN: 0254-5330, 1572-9338. DOI: 10.1007/s10479-005-5724-z. (Visited on 01/10/2026).

Dosovitskiy, Alexey et al. (Oct. 2020). *An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale*. DOI: 10.48550/arXiv.2010.11929. arXiv: 2010.11929 [cs]. (Visited on 04/11/2025).

Edwards, Ashley D. et al. (May 2018). *Imitating Latent Policies from Observation*. DOI: 10.48550/arXiv.1805.07914. arXiv: 1805.07914 [cs]. (Visited on 01/03/2026).

Gao, Shenyuan et al. (Mar. 2025). *AdaWorld: Learning Adaptable World Models with Latent Actions*. DOI: 10.48550/arXiv.2503.18938. arXiv: 2503.18938 [cs]. (Visited on 10/28/2025).

Graves, Alex (Aug. 2013). *Generating Sequences With Recurrent Neural Networks*. DOI: 10.48550/arXiv.1308.0850. arXiv: 1308.0850 [cs]. (Visited on 12/31/2025).

Ha, David and Jürgen Schmidhuber (Mar. 2018). "World Models". In: DOI: 10.5281/zenodo.1207631. arXiv: 1803.10122 [cs]. (Visited on 04/08/2025).

Hafner, Danijar et al. (Nov. 2018). *Learning Latent Dynamics for Planning from Pixels*. DOI: 10.48550/arXiv.1811.04551. arXiv: 1811.04551 [cs]. (Visited on 12/16/2025).

Hafner, Danijar et al. (Dec. 2019). *Dream to Control: Learning Behaviors by Latent Imagination*. DOI: 10.48550/arXiv.1912.01603. arXiv: 1912.01603 [cs]. (Visited on 04/08/2025).

Hafner, Danijar et al. (Oct. 2020). *Mastering Atari with Discrete World Models*. DOI: 10.48550/arXiv.2010.02193. arXiv: 2010.02193 [cs]. (Visited on 04/11/2025).

Hafner, Danijar et al. (June 2022). *Deep Hierarchical Planning from Pixels*. DOI: 10.48550/arXiv.2206.04114. arXiv: 2206.04114 [cs]. (Visited on 02/15/2026).

Hafner, Danijar et al. (Jan. 2023). *Mastering Diverse Domains through World Models*. DOI: 10.48550/arXiv.2301.04104. arXiv: 2301.04104 [cs]. (Visited on 04/17/2025).

Hansen, Nicklas, Hao Su, and Xiaolong Wang (Oct. 2023). *TD-MPC2: Scalable, Robust World Models for Continuous Control*. DOI: 10.48550/arXiv.2310.16828. arXiv: 2310.16828 [cs]. (Visited on 12/09/2025).

He, Kaiming et al. (Dec. 2015). *Deep Residual Learning for Image Recognition*. DOI: 10.48550/arXiv.1512.03385. arXiv: 1512.03385 [cs]. (Visited on 06/29/2025).

He, Kaiming et al. (Nov. 2021). *Masked Autoencoders Are Scalable Vision Learners*. DOI: 10.48550/arXiv.2111.06377. arXiv: 2111.06377 [cs]. (Visited on 12/27/2025).

Higgins, Irina et al. (Feb. 2017). "Beta-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework". In: *International Conference on Learning Representations*. (Visited on 01/01/2026).

Jiang, Zhengyao et al. (Aug. 2022). *Efficient Planning in a Compact Latent Action Space*. DOI: 10.48550/arXiv.2208.10291. arXiv: 2208.10291 [cs]. (Visited on 01/03/2026).

Kaelbling, Leslie Pack, Michael L. Littman, and Anthony R. Cassandra (May 1998). "Planning and Acting in Partially Observable Stochastic Domains". In: *Artificial Intelligence* 101.1, pp. 99–134. ISSN: 0004-3702. DOI: 10.1016/S0004-3702(98)00023-X. (Visited on 12/21/2025).

Kaiser, Lukasz et al. (Mar. 2019). *Model-Based Reinforcement Learning for Atari*. DOI: 10.48550/arXiv.1903.00374. arXiv: 1903.00374 [cs]. (Visited on 12/23/2025).

Kingma, Diederik P. and Max Welling (Dec. 2013). *Auto-Encoding Variational Bayes*. DOI: 10.48550/arXiv.1312.6114. arXiv: 1312.6114 [stat]. (Visited on 12/09/2025).

Kroese, Dirk et al. (Jan. 2016). "Cross-Entropy Method". In: pp. 326–333. DOI: 10.1007/978-1-4419-1153-7_131.

LeCun, Yann (June 2022). "A Path Towards Autonomous Machine Intelligence Version 0.9.2, 2022-06-27". In.

Micheli, Vincent, Eloi Alonso, and François Fleuret (Sept. 2022). *Transformers Are Sample-Efficient World Models*. DOI: 10.48550/arXiv.2209.00588. arXiv: 2209.00588 [cs]. (Visited on 04/15/2025).

Nikulin, Alexander et al. (Feb. 2025). *Latent Action Learning Requires Supervision in the Presence of Distractors*. DOI: 10.48550/arXiv.2502.00379. arXiv: 2502.00379 [cs]. (Visited on 12/28/2025).

Oord, Aaron van den, Oriol Vinyals, and Koray Kavukcuoglu (Nov. 2017). *Neural Discrete Representation Learning*. DOI: 10.48550/arXiv.1711.00937. arXiv: 1711.00937 [cs]. (Visited on 04/11/2025).

Oquab, Maxime et al. (Apr. 2023). *DINOv2: Learning Robust Visual Features without Supervision*. DOI: 10.48550/arXiv.2304.07193. arXiv: 2304.07193 [cs]. (Visited on 03/17/2025).

Pathak, Deepak et al. (May 2017). *Curiosity-Driven Exploration by Self-supervised Prediction*. DOI: 10.48550/arXiv.1705.05363. arXiv: 1705.05363 [cs]. (Visited on 04/15/2025).

Pu, Yuan et al. (Jan. 2025). *UniZero: Generalized and Efficient Planning with Scalable Latent World Models*. DOI: 10.48550/arXiv.2406.10667. arXiv: 2406.10667 [cs]. (Visited on 04/18/2025).

Razavi, Ali, Aaron van den Oord, and Oriol Vinyals (June 2019). *Generating Diverse High-Fidelity Images with VQ-VAE-2*. DOI: 10.48550/arXiv.1906.00446. arXiv: 1906.00446 [cs]. (Visited on 12/29/2025).

Rubinstein, Reuven (Sept. 1999). "The Cross-Entropy Method for Combinatorial and Continuous Optimization". In: *Methodology And Computing In Applied Probability* 1.2, pp. 127–190. ISSN: 1573-7713. DOI: 10.1023/A:1010091220143. (Visited on 12/09/2025).

Schmidt, Dominik and Minqi Jiang (Dec. 2023). *Learning to Act without Actions*. DOI: 10.48550/arXiv.2312.10812. arXiv: 2312.10812 [cs]. (Visited on 10/28/2025).

Schrittwieser, Julian et al. (Dec. 2020). "Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model". In: *Nature* 588.7839, pp. 604–609. ISSN: 0028-0836, 1476-4687. DOI: 10.1038/s41586-020-03051-4. arXiv: 1911.08265 [cs]. (Visited on 01/01/2026).

Talvitie, Erik (Dec. 2016). *Self-Correcting Models for Model-Based Reinforcement Learning*. DOI: 10.48550/arXiv.1612.06018. arXiv: 1612.06018 [cs]. (Visited on 02/14/2026).

Vaswani, Ashish et al. (June 2017). *Attention Is All You Need*. DOI: 10.48550/arXiv.1706.03762. arXiv: 1706.03762 [cs]. (Visited on 12/09/2025).

Williams, Grady, Andrew Aldrich, and Evangelos Theodorou (Sept. 2015). *Model Predictive Path Integral Control Using Covariance Variable Importance Sampling*. DOI: 10.48550/arXiv.1509.01149. arXiv: 1509.01149 [eess]. (Visited on 01/01/2026).

Williams, Grady et al. (May 2016). "Aggressive Driving with Model Predictive Path Integral Control". In: *2016 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 1433–1440. DOI: 10.1109/ICRA.2016.7487277. (Visited on 02/08/2026).

Ye, Seonghyeon et al. (Oct. 2024a). *Latent Action Pretraining from Videos*. DOI: 10.48550/arXiv.2410.11758. arXiv: 2410.11758 [cs]. (Visited on 06/17/2025).

Ye, Seonghyeon et al. (Oct. 2024b). "Latent Action Pretraining from Videos". In: *The Thirteenth International Conference on Learning Representations*. (Visited on 06/29/2025).

Zbontar, Jure et al. (Mar. 2021). *Barlow Twins: Self-Supervised Learning via Redundancy Reduction*. DOI: 10.48550/arXiv.2103.03230. arXiv: 2103.03230 [cs]. (Visited on 01/02/2026).

Zhang, Lixuan et al. (Oct. 2024). "PreLAR: World Model Pre-training with Learnable Action Representation". In: *Computer Vision – ECCV 2024*. Ed. by Aleš Leonardis et al. Vol. 15081. Cham: Springer Nature Switzerland, pp. 185–201. ISBN: 978-3-031-73336-9 978-3-031-73337-6. DOI: 10.1007/978-3-031-73337-6_11. (Visited on 10/28/2025).

Zhou, Gaoyue et al. (Nov. 2024). *DINO-WM: World Models on Pre-trained Visual Features Enable Zero-shot Planning*. DOI: 10.48550/arXiv.2411.04983. arXiv: 2411.04983 [cs]. (Visited on 03/16/2025).

Zhou, Jinghao et al. (Nov. 2021). *iBOT: Image BERT Pre-Training with Online Tokenizer*. DOI: 10.48550/arXiv.2111.07832. arXiv: 2111.07832 [cs]. (Visited on 12/27/2025).

A Appendix

A.1 Implementation and Integration Effort

A large proportion of the total time invested in this thesis was dedicated to implementation and integration work. The research contributions required modifications to the existing codebase as well as the introduction of several new components.

To integrate latent action representations into the pipeline, architectural changes were necessary. Due to the removal of the proprioceptive signal and inefficient data slicing that slowed data loading into the world model, the dataset modules (`pusht_dset.py`, `traj_dset.py`) were refactored.

A new action decoder was introduced as a standalone module (`action_decoder.py`), together with its dedicated training pipeline (`train_action_decoder.py`). Supporting multiple action representations and modifying the loss computation required changes to the world model implementation (`visual_world_model.py`).

In addition, a new exponential moving average (EMA) vector-quantization codebook module (`vq_ema.py`) was implemented, alongside a discrete Cross-Entropy Method (CEM) planner variant (`discrete_cem.py`). The original CEM planner (`cem.py`) was also modified.

The evaluator was extended to support latent action decoding and enhanced with additional diagnostic functionality. A dedicated validation metrics module (`eval_val_metrics.py`) was also implemented.

The planning pipeline (`plan.py`) was modified to correctly load and route both ground-truth and latent action representations. The main training pipeline (`train.py`) was refactored to improve determinism (via improved seeding and deterministic data loading) and to enhance checkpoint handling (state-based loading instead of full-module loading). In addition, the inverse dynamics model (IDM), the VQ codebook, and their respective optimizers were integrated. The loss computation and action routing were adjusted accordingly, and the action diagnostics used in this thesis were integrated into the training and validation workflows. Finally, we provide instructions to reproduce the results.